

TRADING EDGE INTELLIGENCE SYSTEM

DOKUMEN SPESIFIKASI PENGEMBANGAN APLIKASI BERBASIS FITUR (FDD)

Analisis komprehensif, spesifikasi fungsional, skema database, API endpoint, dan petunjuk implementasi Agentic AI untuk pengembangan modular sistem Trading Edge Intelligence System (TEIS).

DIPERSIAPKAN SEBAGAI PERAN:

- Senior Business Analyst & Product Manager (Ruang Lingkup & Alur Bisnis)
- System Analyst & Software Architect (Skema DB, API, Struktur Modular)
- Senior Software Engineer (Best Practices, Validasi, Penanganan Error)

Versi Dokumen: 1.4 (Sesuai Audit Mockup & Skema Database)

Tanggal Pembuatan: 20 Juli 2026

DAFTAR ISI & STRUKTUR DOKUMEN

Dokumen ini memuat spesifikasi teknis lengkap untuk Trading Edge Intelligence System (TEIS). Setiap fitur didokumentasikan secara mandiri dengan alur sebagai berikut:

Analisis Fitur -> Ringkasan & Alur Bisnis -> Dokumentasi Teknis -> Prompt AI -> Catatan Implementasi

Berikut adalah daftar fitur yang tercakup dalam dokumen spesifikasi ini:

Tahap 1 - Analisis Dokumentasi & Prinsip Desain

Tahap 2 - Identifikasi Seluruh Fitur

Tahap 3 - Spesifikasi Fitur Secara Berurutan:

- Fitur 1: Autentikasi & Manajemen Rahasia (Auth & Secret Management)
- Fitur 2: Sinkronisasi & Polling Posisi Binance (Binance Position Polling & Sync)
- Fitur 3: Tangkap Cepat Jurnal (Quick-Tag Capture Web App)
- Fitur 4: Pengumpul Konteks Pasar (Market Context Collector)
- Fitur 5: Manajemen Koleksi & Penggabungan Trade (Trade Collection & Linking)
- Fitur 6: Eksekusi & Batasan Immutability (Trade Execution & Immutability)
- Fitur 7: Pengelola Gambar (Screenshot Manager)
- Fitur 8: Layanan Notifikasi Multi-Saluran (Multi-Channel Notification Service)
- Fitur 9: Wizard Impor Historis (Historical Import Wizard)
- Fitur 10: Layanan Snapshot Ekuitas (Equity Snapshot Service)
- Fitur 11: Mesin Analitis (Analytics Engine)
- Fitur 12: Mesin Penemu Edge (Edge Discovery Engine)
- Fitur 13: Mesin Validasi & Pemantau Status Edge (Edge Validation & Status Monitor)
- Fitur 14: Asisten AI (AI Coach Service)
- Fitur 15: Dasbor & Explorer Cetak Biru Edge (Dashboard & Edge Blueprint Explorer)

TEKNOLOGI STACK SISTEM:

- Backend: Python 3.12 (FastAPI untuk REST API, Celery + Redis untuk Background Job & Scheduler)
- Database & ORM: MySQL 8 (InnoDB, UTF8MB4), SQLAlchemy 2.x, Migrasi Alembic
- Frontend & Storage: React (Vite, Responsive Web App), MinIO (S3-Compatible Object Storage)

TAHAP 1 - ANALISIS DOKUMENTASI & PRINSIP DESAIN

Trading Edge Intelligence System (TEIS) dirancang sebagai sistem pencatatan (journaling) dan analitik personal untuk trading manual futures crypto di Binance. Sistem ini bersifat closed-loop, di mana hasil analitik akan membantu trader memvalidasi edge kualitatif mereka, dan data trade baru akan terus menyempurnakan statistik tersebut.

Prinsip Desain Utama:

- Non-intrusive capture: Trader harus dapat mengisi Quick-Tag dalam waktu <15 detik.
- Immutable snapshot: Data subjektif (setup, psikologi) akan dikunci permanen setelah window koreksi 60 detik berlalu.
- Separation of concerns: Sistem terpisah total dari bot trading lainnya.
- Insight, bukan sinyal: Menampilkan statistik historis-deskriptif, bukan instruksi transaksi otomatis.
- Satuan R: Mengukur performa murni dalam satuan R-multiple untuk memisahkan keputusan sizing dari kualitas setup.

Pemisahan Data Objektif vs Subjektif:

Sistem secara tegas memisahkan data objektif (data pasar nyata dari Binance API seperti entry/exit price, fee, klines) dari data subjektif (penilaian kualitatif trader seperti setup yang digunakan, bias arah mental, dan status emosi). Hal ini penting untuk menghindari hindsight bias (kecenderungan merasa sudah mengetahui hasil akhir sebelum trade selesai).

Bagian yang Membutuhkan Klarifikasi (Needs Clarification) & Rekomendasi:

1. Kriteria Trend HTF/LTF: EMA50 di HTF (4 Jam) dan LTF (1 Jam). Rekomendasi: Gunakan slope kemiringan EMA50 dari 3 candlestick terakhir.
2. Uji Robustness Edge: Rekomendasi: Geser target profit dan stop loss sebesar +/- 5% dan +/- 10% untuk memverifikasi kestabilan expectancy.
3. FDR Target Rate: Target FDR 5-10%. Rekomendasi: Jadikan target FDR sebagai parameter konfigurasi default di angka 5%.

TAHAP 2 - IDENTIFIKASI SELURUH FITUR

Berdasarkan dokumen perencanaan dan audit desain teknis, diidentifikasi 15 fitur pengembangan berikut:

- Fitur 1: Autentikasi & Manajemen Rahasia (Auth & Secret Management)
- Fitur 2: Sinkronisasi & Polling Posisi Binance (Binance Position Polling & Sync)
- Fitur 3: Tangkap Cepat Jurnal (Quick-Tag Capture Web App)
- Fitur 4: Pengumpul Konteks Pasar (Market Context Collector)
- Fitur 5: Manajemen Koleksi & Penggabungan Trade (Trade Collection & Linking)
- Fitur 6: Eksekusi & Batasan Immutability (Trade Execution & Immutability)
- Fitur 7: Pengelola Gambar (Screenshot Manager)
- Fitur 8: Layanan Notifikasi Multi-Saluran (Multi-Channel Notification Service)
- Fitur 9: Wizard Impor Historis (Historical Import Wizard)
- Fitur 10: Layanan Snapshot Ekuitas (Equity Snapshot Service)
- Fitur 11: Mesin Analitis (Analytics Engine)
- Fitur 12: Mesin Penemu Edge (Edge Discovery Engine)

- Fitur 13: Mesin Validasi & Pemantau Status Edge (Edge Validation & Status Monitor)
- Fitur 14: Asisten AI (AI Coach Service)
- Fitur 15: Dasbor & Explorer Cetak Biru Edge (Dashboard & Edge Blueprint Explorer)

FITUR 1 - AUTENTIKASI & MANAJEMEN RAHASIA (AUTH & SECRET MANAGEMENT)

A. Ringkasan Fitur

Fitur ini menyediakan mekanisme login pengguna tunggal (single-user) yang aman dengan Argon2 hashing, verifikasi Dua Faktor (2FA TOTP), serta enkripsi/dekripsi kunci API Binance (AES-256) menggunakan secret key yang disimpan di environment variable server.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Autentikasi & Manajemen Rahasia

2. Tujuan

Mengamankan akses ke aplikasi TEIS karena bersifat personal dan mengamankan kunci API Binance yang sensitif.

3. Deskripsi

Fitur ini mencakup otentikasi login, TOTP 2FA, enkripsi kunci API Binance dengan AES-256, dan pembersihan kredensial dari log audit.

4. Business Flow

1. Pengguna membuka web app.
2. Masuk menggunakan password (diverifikasi dengan Argon2).
3. Pengguna dimintai kode 2FA TOTP.
4. Setelah masuk, pengguna dapat menyimpan atau mengupdate API Key & Secret Binance.
5. API key dienkripsi di backend menggunakan AES-256 (key enkripsi diambil dari environment variable) dan disimpan di database.

5. Aktor

Trader (Single-User)

6. Hak Akses

Akses Penuh (Administrator)

7. Input

- username (String) - Validasi: Alphanumeric, 3-20 karakter (Wajib)
- password (String) - Validasi: Min 8 karakter, campuran huruf & angka (Wajib)
- totp_code (String) - Validasi: Numeric, tepat 6 karakter (Opsional)
- binance_api_key (String) - Validasi: Alphanumeric, panjang sesuai format Binance (Opsional)
- binance_api_secret (String) - Validasi: Alphanumeric, panjang sesuai format Binance (Opsional)

8. Output

JWT Token untuk otentikasi sesi, Status koneksi API Binance.

9. Business Rules

- Kredensial tidak boleh ditulis langsung di kode sumber.

- Kunci enkripsi AES-256 disimpan di file .env dan masuk dalam .gitignore.
- API key Binance hanya memiliki izin read-only. Penarikan dana (withdraw) dan spot trading dinonaktifkan.

10. Validasi

- Validasi kecocokan TOTP sebelum memberikan JWT token.
- Enkripsi harus menggunakan IV (Initialization Vector) unik per baris data.

11. Edge Cases

- Pengguna kehilangan kode 2FA TOTP. Solusinya adalah menyediakan reset manual via command-line script di server.
- Waktu server dan HP pengguna tidak sinkron sehingga TOTP gagal.

12. Error Handling

- HTTP 401 Unauthorized untuk password/TOTP salah.
- HTTP 400 Bad Request jika format API key tidak sesuai.
- Logging percobaan login gagal tanpa menulis password ke log.

13. Database

- Tabel `users` (id, username, password_hash, totp_secret, created_at).
- Tabel `api\_credentials` (id, service_name, encrypted_api_key, encrypted_api_secret, created_at).

14. API

- POST /api/v1/auth/login (Request: username, password; Response: session_id, require_2fa)
- POST /api/v1/auth/verify-2fa (Request: totp_code; Response: access_token)
- POST /api/v1/settings/binance-key (Request: api_key, api_secret; Response: status)

15. UI/UX

- Halaman Login (src/pages/Login.jsx): Minimalis, fokus pada keamanan.
- Halaman Verifikasi 2FA (src/pages/Verify2FA.jsx): Tampilan input 6 kotak angka otomatis fokus.
- Halaman Settings (src/pages/Settings.jsx): Input API key terlindungi (disembunyikan dengan opsi tampilkan/sembunyikan).

16. Security

Enkripsi password menggunakan Argon2id. API key dienkripsi dengan AES-256-GCM. Session menggunakan JWT token dengan masa kedaluwarsa 24 jam.

17. Dependencies

Tidak ada.

18. Acceptance Criteria

- Pengguna hanya dapat mengakses sistem setelah melalui verifikasi password dan TOTP.
- API key disimpan dalam bentuk terenkripsi di database, tidak dapat dibaca langsung oleh admin database tanpa kunci deskripsi.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur AUTENTIKASI & MANAJEMEN RAHASIA (Auth & Secret Management). Tujuan fitur:
Mengamankan akses masuk ke aplikasi personal TEIS (single-user) dan melindungi kunci API Binance yang sensitif menggunakan enkripsi yang kuat di backend.
Fungsi utama:
- Menyediakan formulir login terproteksi Argon2.
- Mengaktifkan verifikasi TOTP 2FA berbasis waktu menggunakan standard RFC 6238.
- Melakukan enkripsi dan dekripsi Binance API Key dan Secret menggunakan algoritma AES-256-GCM.
- Menyediakan halaman pengaturan kredensial terproteksi token otentikasi di frontend.
Alur bisnis:
1. Pengguna memicu login di UI dengan mengirimkan username dan password ke REST API.
2. Backend FastAPI memverifikasi password terhadap hash Argon2id di database (atau .env).
3. Jika password cocok, backend memverifikasi kode TOTP 6 digit yang dikirimkan.
4. Jika 2FA sukses, backend memancarkan JWT Access Token (masa kedaluwarsa 24 jam).
5. Di halaman Pengaturan, trader memasukkan Binance API Key dan API Secret.
6. Backend mengenkripsi kredensial tersebut dengan kunci enkripsi AES-256-GCM dari .env, menghasilkan data byte acak dan tag autentikasi, lalu menyimpannya di DB.
7. Saat polling berjalan, modul sinkronisasi mendekripsi API Key/Secret ini di RAM untuk melakukan koneksi ke Binance.
Validasi:
- Username wajib alphanumeric dan password minimal 8 karakter.
- Kunci enkripsi AES-256 harus tepat 32 byte (256 bit) yang dimuat dari environment variable ENCRYPTION_KEY.
- Token JWT harus divalidasi keabsahannya di setiap request API melalui middleware otentikasi.
Hak akses:
- Hanya satu akun Trader terautentikasi (JWT session).
Kondisi gagal & Penanganan Error:
- Password atau TOTP salah: Kembalikan HTTP 401 Unauthorized, catat percobaan gagal di log audit, jangan crash.
- Kunci enkripsi .env kosong atau tidak valid: Lempar fatal error saat start aplikasi, blokir akses ke Settings API.
Integrasi:
- Input: JSON payload via POST /api/v1/auth/login dan POST /api/v1/settings/binance-key.
- Output: Encrypted DB records, JWT token, status integrasi Binance API.
- Konfigurasi: .env (JWT_SECRET, ENCRYPTION_KEY, ADMIN_PASSWORD_HASH, TOTP_SECRET).
- Audit/Penyimpanan: Tabel `users` dan `api\_credentials`.
Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.
Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan di frontend React (src/pages/Login.jsx, src/pages/Verify2FA.jsx, src/pages/Settings.jsx) dan backend FastAPI (app/api/auth.py, app/models/models.py).

D. Catatan Implementasi

Risiko: Kehilangan TOTP secret dapat mengunci pengguna keluar. Sediakan script Python independen di folder scripts/ untuk mereset password dan 2FA via terminal CLI.

FITUR 2 - SINKRONISASI & POLLING POSISI BINANCE (BINANCE POSITION POLLING & SYNC)

A. Ringkasan Fitur

Fitur ini berjalan secara background menggunakan Celery Beat untuk melakukan polling posisi aktif di Binance Futures setiap 30-60 detik. Ketika posisi baru terdeteksi, sistem membuat 'trade shell' berstatus 'pending_tag' dan mencari fill entry untuk dipasangkan.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Sinkronisasi & Polling Posisi Binance

2. Tujuan

Mendeteksi secara otomatis kapan trader membuka dan menutup posisi di Binance Futures.

3. Deskripsi

Layanan polling berkala menggunakan Celery untuk memantau posisi aktif di Binance Futures (USDT-M), mencatat data entry dan exit fill, dan menyimpannya ke database.

4. Business Flow

1. Celery Beat memicu task `poll\_open\_positions` setiap 30 detik.
2. Task memanggil GET /fapi/v2/positionRisk dari Binance.
3. Jika ada posisi aktif (size != 0) yang belum tercatat di database (trades aktif dengan exit_time IS NULL):
 - a. Buat baris baru di tabel `trades` dengan status `pending\_tag`.
 - b. Tarik histori fill untuk trade pembukaan posisi melalui GET /fapi/v1/userTrades.
 - c. Simpan fill ke `exchange\_fills` dan hubungkan sebagai entry fill di `trade\_fills`.
 - d. Periksa order aktif (GET /fapi/v1/openOrders) untuk mendeteksi order stop_loss (SL) dan take_profit (TP), lalu isi kolom `stop\_loss` dan `take\_profit` di tabel `trades`.
 - e. Picu Notification Service.
4. Jika posisi aktif yang tercatat sebelumnya sudah hilang dari respon positionRisk:
 - a. Tarik histori fill penutupan posisi melalui GET /fapi/v1/userTrades.
 - b. Simpan fill ke `exchange\_fills` dan hubungkan sebagai exit fill di `trade\_fills`.
 - c. Update data penutupan trade (exit_price, exit_time, pnl, fee, rr_realized).
 - d. Pemicuan analisis lanjutan.

5. Aktor

Binance Sync Service (Automated)

6. Hak Akses

Sistem (Background Service)

7. Input

- symbol (String) - Validasi: Format Binance (e.g. BTCUSDT) (Wajib)
- timestamp (DateTime) - Validasi: ISO Format (Wajib)

8. Output

Baris data baru di tabel `trades`, `exchange\_fills`, dan `trade\_fills`.

9. Business Rules

- Polling interval diatur di 30-60 detik untuk meminimalkan beban rate-limit API Binance.
- Sizing/leverage didapatkan langsung dari Binance.
- Duplikasi data dihindari dengan validasi UNIQUE KEY pada `symbol` dan `binance\_trade\_id`.

10. Validasi

- Validasi bahwa fill entry dipetakan sebagai role='entry' dan fill exit dipetakan sebagai role='exit'.
- Periksa apakah timestamp fill logis (entry_time < exit_time).

11. Edge Cases

- API key tidak valid atau expired. Solusinya: kirim notifikasi via Notification Service dan nonaktifkan job sync sementara.
- Posisi ditutup secara manual dalam waktu kurang dari 30 detik (scalping cepat) sehingga entry & exit terdeteksi bersamaan.

12. Error Handling

- Kirim notifikasi kegagalan sinkronisasi jika gagal berturut-turut > 3 kali.
- Log error API dengan format terstruktur JSON.

13. Database

- Mengisi tabel `trades`, `exchange\_fills`, `trade\_fills`.
- Memperbarui status dan timestamps.

14. API

- Panggilan keluar ke Binance Futures API: /fapi/v2/positionRisk, /fapi/v1/userTrades, /fapi/v1/openOrders.

15. UI/UX

- Frontend (src/components/Navbar.jsx): Menampilkan counter/badge notifikasi merah menyala jika ada trade pending_tag.
- Terdapat status indikator koneksi API di footer dashboard.

16. Security

API Key disimpan terenkripsi di DB dan didekripsi hanya di memory backend saat runtime sinkronisasi.

17. Dependencies

Fitur 1 (Autentikasi & Manajemen Rahasia) untuk kredensial API.

18. Acceptance Criteria

- Posisi baru di Binance Futures terdeteksi maksimal dalam 60 detik.
- Trade shell berhasil dibuat di database dengan status pending_tag.
- Kunci SL dan TP rencana terisi otomatis jika dipasang di Binance.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur SINKRONISASI & POLLING POSISI BINANCE (Binance Position Polling & Sync).

Tujuan fitur:

Mendeteksi secara otomatis pembukaan dan penutupan posisi aktif di Binance Futures (USDT-M) secara read-only tanpa eksekusi otomatis, lalu merekamnya ke dalam database sebagai baseline jurnal.

Fungsi utama:

- Melakukan polling posisi aktif dari Binance API `/fapi/v2/positionRisk`` setiap 30-60 detik secara asinkron.
- Membuat record trade baru berstatus 'pending_tag' begitu posisi baru terdeteksi.
- Mengambil detail fill transaksi (harga, kuantitas, fee komisi) dari `/fapi/v1/userTrades`` saat posisi dibuka dan ditutup.
- Membaca order stop loss dan take profit aktif dari `/fapi/v1/openOrders``.

Alur bisnis:

1. Task Celery Beat `'poll_open_positions`` berjalan setiap 30 detik.
2. Dapatkan API credentials dari DB, dekripsi dengan kunci enkripsi AES, panggil GET `/fapi/v2/positionRisk``.
3. Loop setiap item posisi. Jika posisi aktif (`'positionAmt != 0``) dan simbol belum tercatat di database trades aktif (`'exit_time IS NULL``):
 - a. Buat baris baru di tabel `'trades`` dengan status implicit pending_tag (data subjektif kosong).
 - b. Panggil GET `/fapi/v1/userTrades`` untuk simbol terkait pada timestamp pembukaan, ambil fill transaksi, simpan ke `'exchange_fills``, dan buat record penghubung di `'trade_fills`` dengan `'role = 'entry``.
 - c. Panggil GET `/fapi/v1/openOrders`` untuk simbol tersebut. Cari order bertipe `'STOP_MARKET`` (sebagai stop_loss) dan `'TAKE_PROFIT_MARKET`` (sebagai take\_profit), update nilai `'stop_loss`` dan `'take_profit`` di tabel `'trades``.
 - d. Picu asinkron event notifikasi 'trade_pending_tag'.
4. Jika posisi aktif yang tercatat sebelumnya sudah hilang dari response `/fapi/v2/positionRisk`` (`Amt == 0``):
 - a. Panggil GET `/fapi/v1/userTrades`` untuk mengambil fill transaksi penutupan.
 - b. Simpan ke `'exchange_fills`` dan hubungkan ke `'trade_fills`` dengan `'role = 'exit``.
 - c. Perbarui record trade: isi `'exit_price`` (harga rata-rata vwap penutupan), `'exit_time``, `'pnl``, `'fee``, dan hitung `'rr_realized``.
 - d. Ubah status trade menjadi lengkap dan picu pengumpulan market context.

Validasi:

- Pastikan tidak ada data duplikat fill dengan menerapkan UNIQUE KEY (`'symbol``, `'binance_trade_id``) pada tabel `'exchange_fills``.
- Timestamp eksekusi exit harus lebih besar dari entry.

Hak akses:

- Komponen backend internal (Celery background worker).

Kondisi gagal & Penanganan Error:

- Kredensial Binance salah/expired (HTTP 401): Berikan warning ke notification log, nonaktifkan Celery polling task, kirim email darurat ke pengguna.
- Binance API Down/Timeout: Terapkan penundaan retry dengan exponential backoff.

Integrasi:

- Input: Binance Futures API (`/fapi/v2/positionRisk``, `/fapi/v1/userTrades``, `/fapi/v1/openOrders``).
- Output: Record baru di `'trades``, `'exchange_fills``, `'trade_fills``.
- Konfigurasi: celery_app config untuk periodik task.
- Audit/Penyimpanan: Simpan response JSON Binance mentah ke kolom `'exchange_fills.raw_payload`` untuk debugging historis.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Risiko: API Rate limit Binance (2400 weight/menit). Gunakan client yang secara otomatis membaca header response rate-limit dan patuhi batasannya.

FITUR 3 - TANGKAP CEPAT JURNAL (QUICK-TAG CAPTURE WEB APP)

A. Ringkasan Fitur

Formulir input berbasis web responsif yang dioptimalkan untuk pengisian cepat (<15 detik). Digunakan oleh trader untuk mengisi data subjektif (setup, bias arah, emosi, plan) segera setelah entry, sebelum trade dikunci secara permanen setelah 60 detik.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Tangkap Cepat Jurnal

2. Tujuan

Mempermudah pencatatan data subjektif tanpa mengganggu aktivitas trading (non-intrusive).

3. Deskripsi

Formulir interaktif berbasis web (tap-based) untuk melengkapi trade shell yang berstatus pending_tag.

4. Business Flow

1. Trader membuka web app dan melihat badge notifikasi trade baru.
2. Trader membuka form Quick-Tag.
3. Trader memilih setup (multi-checkbox), bias_arah_manual (dropdown), sesi (auto-detect, editable), confidence_level (slider 1-10), kondisi psikologis (tap-based), kepatuhan plan (toggle).
4. Pengguna menekan tombol Simpan.
5. Data tersimpan di database dan masuk ke dalam window koreksi 60 detik.
6. Setelah 60 detik, record dikunci (locked_at diisi) secara otomatis.

5. Aktor

Trader

6. Hak Akses

Trader (Authenticated)

7. Input

- trade_id (UUID) - Validasi: Valid UUID di tabel trades (Wajib)
- setup (List of UUIDs) - Validasi: Referensi valid ke setup_taxonomy_versions (Wajib)
- bias_arah_manual (Enum) - Validasi: bull_trend, bear_trend, range (Wajib)
- session (Enum) - Validasi: asia, london, new_york (Wajib)
- confidence_level (Integer) - Validasi: 1 s.d. 10 (Wajib)
- psychological_tags (JSON List) - Validasi: List of strings (Wajib)
- plan_adherence (Boolean) - Validasi: True / False (Wajib)
- free_notes (Text) - Validasi: Opsional, catatan bebas (Opsional)
- order_type (Enum) - Validasi: limit, market (Wajib)
- screenshot_before_entry (File) - Validasi: Format gambar (PNG/JPG), maks 5MB (Opsional)

8. Output

Pembaruan record trade di database, pengubahan status trade shell menjadi aktif.

9. Business Rules

- Target pengisian <15 detik (tap-based).
- Begitu disimpan, data masuk ke window koreksi 60 detik. Setelah 60 detik, locked_at terisi dan data subjektif tidak dapat diubah/dihapus secara langsung.

10. Validasi

- Validasi bahwa trade_id benar-benar ada dan belum ter-tag.
- Validasi range confidence_level 1-10.

11. Edge Cases

- Pengguna menutup browser sebelum menekan simpan. Solusi: data tetap tersimpan sebagai trade shell berstatus pending_tag di dashboard.

12. Error Handling

- HTTP 400 Bad Request jika field wajib kosong.
- HTTP 409 Conflict jika mencoba mengupdate trade yang statusnya sudah terkunci.

13. Database

- Update tabel `trades` (locked_at).
- Insert tabel `trade\_setup\_tags`, `psychology`, `trade\_execution` (order_type), dan `screenshots` (stage='before_entry').

14. API

- GET /api/v1/journal/pending (Mendapatkan trade berstatus pending_tag)
- POST /api/v1/journal/tag (Menyimpan data Quick-Tag)

15. UI/UX

- Frontend (src/pages/QuickTag.jsx): Tampilan mobile-friendly, tanpa scrolling panjang.
- Elemen UI berupa tombol tap besar (seperti pill/tag) untuk meminimalkan pengetikan keyboard.
- Progress bar 60 detik berjalan mundur setelah tombol 'Simpan' ditekan untuk menunjukkan sisa waktu koreksi.

16. Security

Validasi otentikasi JWT pada API endpoint.

17. Dependencies

Fitur 2 (Sinkronisasi & Polling) untuk mendeteksi trade shell.

18. Acceptance Criteria

- Formulir Quick-Tag dapat diisi dan dikirim tanpa lag.
- Setelah 60 detik dari penyimpanan, database trigger secara otomatis mencegah pengubahan data setup dan psikologi.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur TANGKAP CEPAT JURNAL (Quick-Tag Capture Web App) sesuai mockup 13.1.

Tujuan fitur:

Menyediakan formulir input kualitatif interaktif yang sangat cepat (<15 detik) untuk merekam data subjektif psikologi dan setup trader segera setelah entry, sebelum trade terkunci secara permanen guna menghindari hindsight bias.

Fungsi utama:

- Mengambil daftar trade pending_tag yang belum ter-jurnal.
- Menampilkan antarmuka formulir responsif berbasis React dengan interaksi 'tap-based' (tanpa pengetikan keyboard kecuali catatan).
- Menyimpan parameter setup, bias arah manual, sesi, tingkat kepercayaan, kondisi emosi, dan kepatuhan plan.
- Mengunggah berkas screenshot grafik chart sebelum entry ke penyimpanan objek.
- Menerapkan window waktu koreksi 60 detik sebelum mengisi status `locked\_at`.

Alur bisnis:

1. Halaman web frontend React `src/pages/QuickTag.jsx` memanggil GET `/api/v1/journal/pending` untuk mengambil daftar trade shell.
2. Tampilkan panel detail trade (simbol, arah, entry_price, entry_time) yang terisi otomatis dari Binance.
3. Trader melengkapi formulir:
 - a. SETUP: Tap pill button multi-select (Liquidity Sweep, Order Block, FVG, CHOCH, Equal High).
 - b. BIAS ARAH: Pilih Dropdown (Bull, Bear, Range).
 - c. SESI: Auto-detect berdasarkan jam entry, dapat diedit (Asia, London, New York).
 - d. ORDER TYPE: Pilih Limit atau Market.
 - e. CONFIDENCE: Geser slider numerik dari 1 s.d. 10.
 - f. PSIKOLOGI: Tap pill button multi-select (Sesuai Plan, FOMO, Revenge, Lelah, Tenang).
 - g. KEPATUHAN PLAN: Switch toggle ON/OFF.
 - h. SCREENSHOT BEFORE-ENTRY: Seret/unggah file gambar (opsional).
 - i. CATATAN BEBAS: Teks bebas pendek (opsional).
4. Klik tombol 'Simpan'. Kirim request POST ke `/api/v1/journal/tag`.
5. Backend memproses request, menyimpan data ke DB, dan memicu delay task Celery selama 60 detik.
6. Frontend menampilkan progress bar visual berjalan mundur 60 detik. Pengguna dapat menekan 'Edit' untuk memperbaiki data.
7. Setelah 60 detik, task Celery backend mengisi `trades.locked\_at` dengan timestamp saat itu, mengunci data secara permanen.

Validasi:

- Field setup, bias_arah_manual, confidence_level, plan_adherence, dan order_type wajib diisi.
- Jika request update masuk setelah `locked\_at` terisi, backend harus melempar error HTTP 409 Conflict.

Hak akses:

- Trader terotentikasi (JWT Token).

Kondisi gagal & Penanganan Error:

- Upload file bukan gambar atau > 5MB: Tolak di level frontend dan backend, tampilkan pesan warning.
- Endpoint offline saat submit: Tulis data ke IndexedDB sementara (offline-first fallback) jika koneksi internet mendadak hilang, kirim otomatis begitu online.

Integrasi:

- Input: Request multipart/form-data dari frontend.
- Output: Record baru di `trade\_setup\_tags`, `psychology`, `trade\_execution`, dan `screenshots`.
- Konfigurasi: 60 detik correction window.
- Audit/Penyimpanan: Simpan path file gambar ke tabel `screenshots`.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Risiko: Hindsight bias jika data diisi terlambat. Pastikan sistem segera menampilkan banner alert di frontend ketika trade shell

baru terdeteksi oleh backend.

FITUR 4 - PENGUMPUL KONTEKS PASAR (MARKET CONTEXT COLLECTOR)

A. Ringkasan Fitur

Fitur background service yang mengumpulkan metrik pasar objektif saat trade terjadi (ATR, volume 24 jam, BTC dominance, Fear & Greed Index, open interest, funding rate) dan menghitung trend HTF/LTF berbasis EMA50 secara otomatis.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Pengumpul Konteks Pasar

2. Tujuan

Menyediakan metrik pasar objektif yang lengkap saat trade dibuka untuk analisis korelasi.

3. Deskripsi

Mengambil data pasar dari Binance API dan API pihak ketiga (Alternative.me, CoinGecko) pada saat entry, menghitung trend EMA50, dan menyimpannya di tabel `market_context`.

4. Business Flow

1. Quick-Tag tersimpan.
2. Backend memicu task `collect_market_context`` di Celery.
3. Task mengambil data candlestick (klines) 4 Jam (HTF) dan 1 Jam (LTF) dari Binance.
4. Hitung EMA50 untuk HTF dan LTF. Tentukan trend: 'bull' jika harga tutup > EMA50 dan slope EMA50 naik (dari 3 candle terakhir), 'bear' jika sebaliknya, 'range' jika berosilasi.
5. Ambil data ATR, volume 24 jam, open interest, dan funding rate.
6. Ambil Fear & Greed Index terbaru dari alternative.me.
7. Ambil BTC dominance terbaru dari CoinGecko.
8. Simpan snapshot metrik ke tabel `market_context``.

5. Aktor

Market Context Collector (Automated)

6. Hak Akses

Sistem (Background Service)

7. Input

- `trade_id` (UUID) - Validasi: Valid UUID di tabel `trades` (Wajib)

8. Output

Record baru di tabel `market_context`` yang terhubung ke `trade_id`.

9. Business Rules

- Trend dihitung secara otomatis berdasarkan harga historis, bukan tebakan subjektif.
- Data eksternal non-Binance (Fear & Greed, BTC dominance) disinkronkan berkala per jam secara pasif untuk menghindari rate-limit API publik.

10. Validasi

- Validasi bahwa trade_id valid dan belum memiliki market context.
- Presisi numerik untuk funding_rate dan open_interest menggunakan DECIMAL.

11. Edge Cases

- API CoinGecko atau Alternative.me down. Solusinya: gunakan data terakhir yang di-cache di sistem.

12. Error Handling

- Log error koneksi eksternal.
- Simpan data seadanya jika salah satu API eksternal gagal merespon.

13. Database

- Mengisi tabel `market\_context`.

14. API

- GET /fapi/v1/klines (Binance klines)
- GET /futures/data/openInterestHist (Binance open interest)
- GET /fapi/v1/fundingRate (Binance funding rate)
- GET alternative.me/fng/ (Fear & Greed Index)
- GET coingecko.com/api/v3/global (BTC Dominance)

15. UI/UX

- Menampilkan data Market Context di kartu detail trade (Mockup 13.3) di komponen frontend src/components/MarketContextCard.jsx.

16. Security

Panggilan ke API publik tidak memerlukan tanda tangan (signature) API key Binance.

17. Dependencies

Fitur 3 (Quick-Tag) sebagai pemicu pengumpulan context.

18. Acceptance Criteria

- Market context tersimpan otomatis setelah Quick-Tag dikirim.
- Nilai trend_hf dan trend_lf sesuai dengan perhitungan matematis EMA50.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur PENGUMPUL KONTEKS PASAR (Market Context Collector).
Tujuan fitur:
Mengumpulkan seluruh indikator kondisi pasar objektif (market context) secara otomatis pada detik ketika posisi entry dicatat, serta mengkalkulasikan status tren EMA50.
Fungsi utama:
- Mengambil data candlestick (klines) untuk timeframes 4 Jam (HTF) dan 1 Jam (LTF) dari Binance.
- Mengkalkulasikan nilai EMA50 (Exponential Moving Average) dan Average True Range (ATR).
- Menghitung arah kemiringan (slope) tren secara matematis.
- Menarik indikator makro eksternal (BTC Dominance & Fear & Greed Index) menggunakan mekanisme caching.
- Menyimpan snapshot data context ke tabel `market\_context` di database MySQL.
Alur bisnis:
1. Begitu Quick-Tag berhasil disimpan (Fitur 3), picu task Celery asinkron `collect\_market\_context(trade\_id)`.
2. Dapatkan objek trade dari DB, lalu panggil GET `/fapi/v1/klines` dari Binance untuk simbol trade terkait pada timeframe 1H dan 4H (tarik minimal 100 candle ke belakang).
3. Hitung indikator teknis:
 a. Hitung nilai EMA50 saat ini.
 b. Hitung slope EMA50: hitung tren 'bull' jika harga penutupan terakhir di atas EMA50 dan `EMA50[t] > EMA50[t-1] > EMA50[t-2]` (slope positif), 'bear' jika sebaliknya, dan 'range' jika tidak memenuhi keduanya.
 c. Hitung ATR-14 menggunakan data klines 1 Jam.
4. Panggil GET `/futures/data/openInterestHist` untuk data open interest.
5. Panggil GET `/fapi/v1/fundingRate` untuk data funding rate.
6. Tarik data BTC Dominance (CoinGecko) dan Fear & Greed Index (Alternative.me) dari cache Redis lokal (jika tidak ada di Redis, panggil API eksternal dan cache selama 1 jam).
7. Simpan seluruh data tersebut sebagai baris baru di tabel `market\_context`.
Validasi:
- Presisi data numerik harus menggunakan presisi tinggi `DECIMAL(20,8)` atau `DECIMAL(10,6)`.
- Jika `trade\_id` tidak ditemukan, hentikan operasi (fail-fast).
Hak akses:
- Komponen backend internal (Celery worker).
Kondisi gagal & Penanganan Error:
- Jika panggilan API CoinGecko/Alternative.me gagal karena rate limit, gunakan data cache terakhir di Redis, jangan gagalkan penyimpanan data Binance.
- Jika data klines Binance malformed, tulis log error ke sentry dan gunakan fallback tren = 'range'.
Integrasi:
- Input: `trade\_id` via internal task parameters.
- Output: Record baru di tabel `market\_context`.
- Konfigurasi: Redis cache setting, API keys untuk CoinGecko (jika ada).
- Audit/Penyimpanan: Tabel `market\_context` terhubung ke `trades`.
Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.
Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Optimasi: Caching data API eksternal sangat penting karena CoinGecko API gratis memiliki rate limit ketat (30 requests/menit).

FITUR 5 - MANAJEMEN KOLEKSI & PENGGABUNGAN TRADE (TRADE COLLECTION & LINKING)

A. Ringkasan Fitur

Fitur ini melakukan penggabungan otomatis data subjektif (Quick-Tag) dan objektif (fills Binance) berdasarkan kedekatan waktu dan simbol pasar. Fitur ini juga menghitung durasi hold, realized RR, dan PnL bersih setelah dikurangi fee.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Manajemen Koleksi & Penggabungan Trade

2. Tujuan

Mengintegrasikan data fill eksekusi Binance dengan data jurnal kualitatif trader.

3. Deskripsi

Fungsi inti yang menggabungkan record trade manual/sync dengan fill Binance sesungguhnya, menghitung parameter performa secara otomatis.

4. Business Flow

1. Posisi Binance dideteksi dan Quick-Tag disimpan.
2. Sistem mengambil entry fill dan exit fill dari exchange_fills.
3. Sistem memasang fill entry/exit ke `trade\_fills`.
4. Sistem melakukan kalkulasi otomatis terhadap data gabungan.
5. Data performa (PnL bersih, realized RR, holding time) diperbarui di tabel `trades`.

5. Aktor

Sistem (Automated)

6. Hak Akses

Sistem (Background Service)

7. Input

- trade_id (UUID) - Validasi: Valid UUID di tabel trades (Wajib)

8. Output

Pembaruan record trade dengan PnL bersih, realized RR, holding time, dan fee di tabel `trades`.

9. Business Rules

- Setiap trade minimal memiliki satu fill pembukaan (role='entry') dan satu fill penutupan (role='exit') di tabel `trade\_fills`.
- PnL Bersih = PnL kotor - total fee - total funding fee dari seluruh fill yang terhubung.
- Realized RR = PnL Bersih / risk_amount.
- Holding Time = TIMESTAMPDIFF(SECOND, entry_time, exit_time).

10. Validasi

- Validasi kecocokan simbol (symbol) antara fill dan trade.
- Deteksi duplikasi fill via constraint unik database.

11. Edge Cases

- Satu trade ditutup dengan beberapa kali fill (multiple fills) dari Binance (karena masalah likuiditas). Solusinya: sistem harus menjumlahkan seluruh qty dan menghitung average price untuk fill entry/exit, lalu memetakan ke trade_fills.

12. Error Handling

- Tandai status linking sebagai `pending sync` jika fill penutupan belum lengkap.

13. Database

- Membaca & menulis: `trades`, `exchange\_fills`, `trade\_fills`.

14. API

- Panggilan API Binance internal `/fapi/v1/userTrades`.

15. UI/UX

- Frontend (src/pages/Journal.jsx): Menampilkan badge status 'Live' atau 'Import' di daftar jurnal (Mockup 13.2).

16. Security

Otentikasi internal API.

17. Dependencies

Fitur 2 (Binance Sync) dan Fitur 3 (Quick-Tag).

18. Acceptance Criteria

- Perhitungan holding_time, pnl, fee, dan rr_realized akurat hingga 8 desimal.
- Data trade tidak memiliki duplikasi fill eksekusi.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

```
Anda adalah Senior Software Engineer. Bangun fitur MANAJEMEN KOLEKSI & PENGGABUNGAN TRADE (Trade Collection & Linking).
Tujuan fitur:
Mengintegrasikan data fill eksekusi dari Binance API dengan data jurnal kualitatif trader, serta mengkalkulasikan metrik performa secara otomatis.
Fungsi utama:
- Mencocokkan data transaksi mentah (`exchange_fills`) dengan data jurnal (`trades`) berdasarkan kedekatan waktu dan simbol.
- Mendukung penggabungan beberapa fill terpisah (multi-fills) menjadi satu harga rata-rata tertimbang (VWAP).
- Menghitung otomatis durasi hold, realized Risk-to-Reward (RR), komisi fee, dan PnL bersih setelah dikurangi biaya funding.
- Menandai status sinkronisasi trade.
Alur bisnis:
1. Begitu task sinkronisasi Binance (Fitur 2) mendapatkan fill baru, jalankan task `link_trade_fills(trade_id)`.
2. Cari semua baris fill di tabel `exchange_fills` yang cocok dengan simbol dan rentang waktu pembukaan/penutupan trade.
3. Hubungkan fill pembukaan ke tabel `trade_fills` dengan `role = 'entry'`, dan fill penutupan dengan `role = 'exit'`.
4. Jika terdapat lebih dari satu fill untuk entry atau exit (multi-fills):
   - Hitung volume-weighted average price (VWAP) sebagai `entry_price` atau `exit_price` di tabel `trades`.
   - Jumlahkan seluruh kuantitas (qty) dan biaya fee dari fill terkait.
5. Hitung metrik keuangan:
   a. PnL Bersih = Jumlah PnL kotor - Total komisi fee - Total funding fee dari seluruh fill yang terhubung.
   b. Realized RR = PnL Bersih / `trades.risk_amount`.
   c. Holding Time = Selisih detik antara `entry_time` dan `exit_time`.
6. Simpan hasil kalkulasi ke tabel `trades` dan ubah status linking jika fill exit sudah terpasang.
Validasi:
- Semua perhitungan keuangan wajib menggunakan kelas `Decimal` Python untuk menghindari precision error.
- Simbol pada trade harus cocok dengan simbol pada exchange_fills.
Hak akses:
- Komponen backend internal.
Kondisi gagal & Penanganan Error:
- Jika `risk_amount` bernilai nol atau null: Gunakan default risk_amount dari konfigurasi untuk mencegah pembagian dengan nol saat menghitung realized RR.
- Jika fill entry tidak ditemukan: Tandai status trade sebagai 'pending sync' dan kirim warning log.
Integrasi:
- Input: `trade_id` dari background worker.
- Output: Update record tabel `trades`.
- Konfigurasi: Default risk amount, parameter time tolerance.
- Audit/Penyimpanan: Tabel `trades`, `trade_fills`, `exchange_fills`.
Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.
Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.
```

D. Catatan Implementasi

Perhatian khusus: Pastikan presisi tinggi menggunakan tipe data `Decimal` di Python dan `DECIMAL(20,8)` di database MySQL untuk mencegah pembulatan yang tidak akurat.

FITUR 6 - EKSEKUSI & BATASAN IMMUTABILITY (TRADE EXECUTION & IMMUTABILITY)

A. Ringkasan Fitur

Fitur ini mengelola detail eksekusi trade (tipe order, trailing stop, breakeven) dan menerapkan penguncian data subjektif (immutability) di level database menggunakan trigger MySQL untuk mencegah hindsight bias.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Eksekusi & Batasan Immutability

2. Tujuan

Menjaga integritas data subjektif jurnal dengan mencegah perubahan data setelah trade selesai (mencegah manipulasi psikologis).

3. Deskripsi

Skema pembatasan update dan delete langsung di database menggunakan database triggers pada tabel psychology dan trade_setup_tags.

4. Business Flow

1. Quick-Tag disimpan.
2. Setelah 60 detik (window koreksi berakhir), `trades.locked\_at` diisi oleh sistem.
3. Jika ada percobaan UPDATE atau DELETE pada data di tabel `psychology` atau `trade\_setup\_tags`:
 - Database trigger memeriksa status `locked\_at` dari trade induk.
 - Jika `locked\_at` IS NOT NULL, trigger membatalkan operasi dan mengembalikan error.
4. Jika trader perlu melakukan perbaikan data pasca-penguncian, mereka harus menginput koreksi melalui mekanisme penulisan log baru ke tabel `trade\_corrections`.

5. Aktor

Trader / Sistem

6. Hak Akses

Trader (Hanya via mekanisme koreksi)

7. Input

- original_trade_id (UUID) - Validasi: Valid UUID di tabel trades (Wajib)
- field_name (String) - Validasi: Nama kolom yang dikoreksi (e.g. confidence_level) (Wajib)
- old_value (String) - Validasi: Nilai lama (Wajib)
- new_value (String) - Validasi: Nilai baru (Wajib)
- reason (Text) - Validasi: Alasan koreksi data (Wajib)

8. Output

Record baru di tabel `trade\_corrections`.

9. Business Rules

- Kolom `locked\_at` hanya boleh diisi sekali oleh sistem dan tidak boleh di-update menjadi NULL.
- Modifikasi langsung ke data subjektif yang telah terkunci dilarang keras di level database (SQL Triggers).

10. Validasi

- Validasi bahwa field_name yang dikoreksi terdaftar dalam taksonomi yang diizinkan.
- Alasan (reason) koreksi wajib diisi minimal 10 karakter.

11. Edge Cases

- Perubahan skema taksonomi tag di masa depan. Solusinya: taksonomi menggunakan tabel versi (`setup\_taxonomy\_versions`).

12. Error Handling

- Trigger mengembalikan SQLSTATE '45000' dengan pesan 'Trade sudah terkunci, gunakan trade_corrections'.

13. Database

- Tabel `trade\_execution` (id, trade_id, order_type, moved_to_breakeven, trailing_stop_used, exit_reason).
- Tabel `trade\_corrections` (id, original_trade_id, field_name, old_value, new_value, reason, corrected_at).
- Triggers MySQL: `before\_update\_psychology`, `before\_delete\_setup\_tags`.

14. API

- POST /api/v1/journal/correct (Melakukan koreksi data subjektif)

15. UI/UX

- Frontend (src/pages/TradeDetail.jsx): Tombol edit diganti dengan tombol 'Ajukan Koreksi' jika status trade terkunci (Mockup 13.3). Menampilkan modal formulir pengajuan koreksi (field nama, nilai lama, nilai baru, alasan).

16. Security

- Autentikasi JWT.
- Enforce immutability di level database (triggers) sehingga bypassing di level kode aplikasi FastAPI tetap akan diblokir oleh MySQL.

17. Dependencies

Fitur 3 (Quick-Tag).

18. Acceptance Criteria

- Percobaan query SQL `UPDATE` langsung ke tabel `psychology` untuk trade yang memiliki `locked\_at` menghasilkan error database.
- Histori koreksi tersimpan lengkap di tabel `trade\_corrections`.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

```
Anda adalah Senior Software Engineer. Bangun fitur EKSEKUSI & BATASAN IMMUTABILITY (Trade Execution & Immutability).
Tujuan fitur:
Mencegah hindsight bias (manipulasi jurnal secara retroaktif) dengan mengunci data subjektif (psikologi, setup) menggunakan trigger di database MySQL, dan menyediakan mekanisme audit log koreksi terkontrol.
Fungsi utama:
- Merekam parameter eksekusi (tipe order, breakeven, trailing stop).
- Menerapkan pembatasan write/delete di database MySQL melalui trigger setelah `locked_at` diisi.
- Menyediakan API endpoint untuk mengajukan koreksi data yang tersimpan di tabel audit `trade_corrections`.
Alur bisnis:
1. Setelah window waktu 60 detik berakhir, sistem menandai trade dengan mengisi `trades.locked_at`.
2. Database MySQL mengaktifkan triggers:
  a. `BEFORE UPDATE` dan `BEFORE DELETE` pada tabel `psychology` dan `trade_setup_tags`.
  b. Trigger memeriksa jika `locked_at` milik trade induk di tabel `trades` bernilai tidak NULL.
  c. Jika terisi, tolak operasi dan kembalikan pesan error.
3. Pengguna yang ingin mengubah data subjektif terpaksa menggunakan tombol 'Ajukan Koreksi' di UI detail trade.
4. UI memanggil API POST `/api/v1/journal/correct` untuk menyimpan entri koreksi (kolom, nilai lama, nilai baru, alasan).
Validasi:
- Field `reason` pada koreksi minimal memiliki 10 karakter.
- Validasi bahwa field yang ingin diubah benar-benar terdaftar di taksonomi database.
Hak akses:
- Hanya Trader terautentikasi (JWT).
Kondisi gagal & Penanganan Error:
- Query UPDATE ilegal ke tabel terkunci: Database melempar error SQLSTATE '45000'. Kode FastAPI menangkap error ini dan menerjemahkannya menjadi HTTP 409 Conflict dengan pesan user-friendly.
Integrasi:
- Input: JSON payload dari `src/components/CorrectionModal.jsx` ke POST `/api/v1/journal/correct`.
- Output: Audit record baru di tabel `trade_corrections`.
- Konfigurasi: Triggers DDL di file migrasi Alembic.
- Audit/Penyimpanan: Tabel `trade_corrections` terhubung ke `trades`.
Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.
Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.
```

D. Catatan Implementasi

Risiko: Trigger database dapat menyebabkan kegagalan sistem jika tidak ditangani dengan benar pada level ORM SQLAlchemy. Pastikan kode API menangkap exception SQLAlchemy `OperationalError` atau `IntegrityError` akibat trigger tersebut.

FITUR 7 - PENGELOLA GAMBAR (SCREENSHOT MANAGER)

A. Ringkasan Fitur

Fitur ini menangani pengunggahan tangkapan layar (screenshot) chart trading ke MinIO object storage. Gambar diunggah pada 3 tahap berbeda: before_entry (saat Quick-Tag), during_trade, dan exit (via Trade Detail) dengan kompresi otomatis.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Pengelola Gambar

2. Tujuan

Menyimpan dokumentasi visual chart untuk membantu review mingguan (Weekly Review).

3. Deskripsi

Layanan penyimpanan objek gambar di MinIO (S3-compatible) yang mencakup kompresi otomatis dan retensi data.

4. Business Flow

1. Trader mengunggah file gambar pada form Quick-Tag atau halaman Trade Detail.
2. Backend menerima file gambar.
3. Backend melakukan kompresi otomatis (menggunakan Pillow) untuk mengurangi ukuran file.
4. File diunggah ke MinIO object storage dengan struktur path `/screenshots/{trade\_id}/{stage}.png`.
5. Path file eksternal disimpan ke tabel `screenshots` di database.

5. Aktor

Trader

6. Hak Akses

Trader (Authenticated)

7. Input

- trade_id (UUID) - Validasi: Valid UUID di tabel trades (Wajib)
- stage (Enum) - Validasi: before_entry, during_trade, exit (Wajib)
- file (Binary File) - Validasi: PNG/JPG/WEBP, maksimal 5MB sebelum kompresi (Wajib)

8. Output

URL atau path file screenshot yang tersimpan, record baru di tabel `screenshots`.

9. Business Rules

- Gambar dikompresi ke format WebP dengan quality=80 untuk menghemat ruang penyimpanan.
- Maksimal 1 screenshot per stage per trade.

10. Validasi

- Validasi tipe mime-type file (hanya gambar).
- Validasi ukuran file maks 5MB.

11. Edge Cases

- MinIO server tidak dapat dihubungi. Solusinya: simpan sementara di folder temp server lokal dan jalankan sync background task ke MinIO.

12. Error Handling

- HTTP 413 Payload Too Large jika file melebihi 5MB.
- Log error integrasi S3.

13. Database

- Tabel `screenshots` (id, trade_id, stage, file_path, uploaded_at).

14. API

- POST /api/v1/screenshots/upload (Multipart/form-data request; Response: file_path)

15. UI/UX

- Frontend (src/components/ImageUploader.jsx): Dropzone area interaktif pada form Quick-Tag.
- Preview thumbnail gambar di halaman Trade Detail (src/pages/TradeDetail.jsx) dengan tombol upload terpisah per stage (Mockup 13.3).

16. Security

Bucket MinIO di-set private. Akses file menggunakan pre-signed URL berdurasi pendek (misal: 15 menit) yang digenerate oleh backend.

17. Dependencies

Fitur 3 (Quick-Tag) dan Fitur 5 (Trade Collection).

18. Acceptance Criteria

- Gambar berhasil diunggah ke MinIO.
- File tersimpan dalam format terkompresi (WebP).
- Halaman detail menampilkan gambar menggunakan pre-signed URL yang aman.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur PENGELOLA GAMBAR (Screenshot Manager).
Tujuan fitur:
Menyediakan penyimpanan tangkapan layar (screenshot) chart trading pada object storage terenkripsi dan melakukan kompresi otomatis untuk menghemat ruang.
Fungsi utama:
- Mengompresi file gambar yang diunggah ke format WebP.
- Mengunggah file ke MinIO object storage dengan skema folder terstruktur.
- Menyimpan metadata path file ke tabel `screenshots` di MySQL.
- Menghasilkan pre-signed URL berdurasi pendek untuk menampilkan gambar di UI secara aman.
Alur bisnis:
1. Trader mengunggah file chart melalui dropzone di UI Quick-Tag (`before\_entry`) atau di UI Trade Detail (`during\_trade`, `exit`).
2. Frontend React `src/components/ImageUploader.jsx` mengirim file menggunakan request multipart/form-data ke POST `/api/v1/screenshots/upload`.
3. Backend memproses file menggunakan Pillow: konversi ke WebP, atur kualitas kompresi ke 80.
4. Unggah file biner WebP ke bucket MinIO privat dengan key `/screenshots/{trade\_id}/{stage}.webp`.
5. Tulis metadata ke tabel `screenshots`.
6. Ketika detail trade dimuat, backend memanggil MinIO SDK untuk menghasilkan pre-signed URL (exp: 15 menit) dan menyisipkannya ke respon API agar frontend dapat merender gambar.
Validasi:
- MIME-type file harus berupa `image/png` atau `image/jpeg`.
- Maksimal ukuran file asal 5MB.
Hak akses:
- Hanya Trader terautentikasi (JWT).
Kondisi gagal & Penanganan Error:
- Koneksi MinIO mati: Kembalikan HTTP 503 Service Unavailable, log error ke sistem pemantau.
Integrasi:
- Input: File upload binary stream via API.
- Output: File tersimpan di MinIO bucket, record baru di database, pre-signed URL di JSON detail.
- Konfigurasi: MinIO credentials (.env), S3 bucket name.
- Audit/Penyimpanan: Tabel `screenshots`.
Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.
Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Optimasi: Gunakan format WebP karena memiliki kompresi jauh lebih baik dibanding PNG untuk tangkapan layar grafik chart, tanpa mengurangi keterbacaan teks/angka di chart.

FITUR 8 - LAYANAN NOTIFIKASI MULTI-SALURAN (MULTI-CHANNEL NOTIFICATION SERVICE)

A. Ringkasan Fitur

Layanan yang mengirimkan alert otomatis ke tiga saluran sekaligus (in-app banner, Web Push, dan email) setiap kali terdeteksi trade baru yang perlu di-tag, atau ketika status edge Production mengalami penurunan performa.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Layanan Notifikasi Multi-Saluran

2. Tujuan

Menarik perhatian trader secepat mungkin agar tidak melewatkan pencatatan Quick-Tag atau ketika sistem mendeteksi edge yang melemah.

3. Deskripsi

Layanan notifikasi real-time yang memancarkan alert secara simultan ke tiga channel dan mencatatnya ke database untuk audit.

4. Business Flow

1. Pemicu aktif (misal: Binance Sync mendeteksi trade baru, atau Edge Validation mendeteksi penurunan status edge).
2. Notification Service dipanggil dengan tipe notifikasi dan konten pesan.
3. Sistem membuat record notifikasi di tabel `system\_notifications`.
4. Sistem mengirimkan alert secara asinkron ke:
 - a. Banner in-app (melalui WebSocket/SSE di frontend).
 - b. Web Push Notification (ke browser HP/Desktop menggunakan Service Worker & kunci VAPID).
 - c. Email (melalui SMTP relay ke email trader).

5. Aktor

Notification Service (Automated)

6. Hak Akses

Sistem (Background Service)

7. Input

- type (Enum) - Validasi: trade_pending_tag, edge_status_change, sync_failure (Wajib)
- reference_id (UUID) - Validasi: UUID trade atau edge_blueprint terkait (Opsional)
- message (Text) - Validasi: Teks pesan notifikasi (Wajib)

8. Output

Notifikasi terkirim di browser, push banner di HP, email di inbox, dan record baru di tabel `system\_notifications`.

9. Business Rules

- Setiap alert dikirim ke tiga channel sekaligus secara paralel (asinkron via Celery).
- Setiap notifikasi dicatat terpisah per baris per channel di tabel system_notifications untuk mempermudah audit kesuksesan pengiriman.

10. Validasi

Validasi kunci VAPID dan alamat email tujuan sebelum melakukan pengiriman.

11. Edge Cases

- Pengguna menolak izin Web Push di browser. Solusinya: sistem tetap mengirim email dan menampilkan banner in-app.
- SMTP relay down. Solusinya: gunakan Celery retry dengan interval penundaan.

12. Error Handling

- Catat error pengiriman di logger dan update status notifikasi di database.
- Jangan biarkan kegagalan notifikasi membatalkan alur bisnis utama (seperti penyimpanan trade).

13. Database

- Tabel `system\_notifications` (id, type, reference_id, channel: 'in_app'/'web_push'/'email', message, sent_at, acknowledged_at).

14. API

- GET /api/v1/notifications (Mendapatkan notifikasi in-app aktif)
- POST /api/v1/notifications/subscribe-push (Menyimpan subscription Web Push dari frontend)

15. UI/UX

- Frontend (src/components/NotificationBanner.jsx): Banner kuning/oranye di navbar web app jika ada trade menunggu tag (Mockup 13.1 & 13.8).
- Bell icon dengan dropdown notifikasi di dasbor.
- Service Worker di frontend untuk menerima push notification di background desktop/mobile OS.

16. Security

Enkripsi kunci VAPID dan credentials SMTP di environment variable.

17. Dependencies

Fitur 2 (Binance Sync) atau Fitur 13 (Edge Validation) sebagai pemicu.

18. Acceptance Criteria

- Notifikasi terkirim secara instan di in-app banner saat trade baru terdeteksi.
- Web push berhasil masuk ke browser bahkan ketika tab aplikasi TEIS ditutup.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur LAYANAN NOTIFIKASI MULTI-SALURAN (Multi-Channel Notification Service).

Tujuan fitur:
Mengirimkan alert pemberitahuan secara real-time ke tiga saluran (in-app banner, Web Push browser, dan email SMTP) secara simultan untuk menarik perhatian trader secepat mungkin.

Fungsi utama:

- Merekam log notifikasi di tabel database `system\_notifications` untuk audit.
- Memancarkan pesan real-time ke frontend via WebSocket connection.
- Mengirim push notification ke HP/browser menggunakan kunci VAPID dan pustaka `pywebpush`.
- Mengirim email pemberitahuan menggunakan SMTP relay asinkron.

Alur bisnis:

1. Pemicu memanggil modul notifikasi dengan payload berisi `type`, `reference\_id`, dan `message`.
2. Simpan 3 baris data baru ke tabel `system\_notifications` (satu untuk masing-masing channel: in_app, web_push, email).
3. Jalankan Celery tasks secara asinkron:
 - a. Task 1: WebSocket Broadcast ke frontend React. Komponen `src/components/NotificationBanner.jsx` menangkap pesan dan merender banner oranye di bagian atas dasbor.
 - b. Task 2: Ambil client push subscription dari DB, panggil `pywebpush` untuk mengirim notifikasi push ke perangkat pengguna. Service worker browser (`public/service-worker.js`) merender banner notifikasi sistem operasi.
 - c. Task 3: Panggil SMTP relay menggunakan `aiosmtplib` untuk mengirim email ke alamat terdaftar.

Validasi:

- Alamat email penerima harus tervalidasi formatnya.
- Kunci VAPID (private dan public key) harus dikonfigurasi di server.

Hak akses:

- Internal backend, otentikasi JWT hanya untuk pendaftaran subscription push oleh pengguna.

Kondisi gagal & Penanganan Error:

- Salah satu channel gagal mengirim (misal: SMTP error): log error spesifik untuk baris notifikasi terkait di DB, jangan batalkan pengiriman channel lainnya.

Integrasi:

- Input: JSON payload dari pemicu internal.
- Output: WebSocket message, Push payload, email SMTP, DB record.
- Konfigurasi: SMTP server host/port, VAPID Keys (.env).
- Audit/Penyimpanan: Tabel `system\_notifications`.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Perhatian developer: Pastikan Service Worker di frontend React dikonfigurasi dengan benar untuk mendengarkan event 'push' dari browser sistem operasi Windows/Android/iOS.

FITUR 9 - WIZARD IMPOR HISTORIS (HISTORICAL IMPORT WIZARD)

A. Ringkasan Fitur

Fitur sekali-jalan (bisa dijalankan ulang secara manual) untuk menarik riwayat trade closed lama dari Binance sebelum TEIS diinstal. Hanya data objektif yang disimpan, sementara data subjektif dibiarkan kosong untuk mencegah hindsight bias.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Wizard Impor Historis

2. Tujuan

Menarik data historis trading masa lalu untuk membangun baseline performa awal (win rate, profit factor).

3. Deskripsi

Fitur impor batch trade dari Binance API secara kronologis mundur dengan visualisasi progress bar.

4. Business Flow

1. Trader membuka menu Import di Dashboard (Mockup 13.9).
2. Trader memilih rentang tanggal impor (misal: 1 Jan 2025 - sekarang).
3. Trader menekan tombol 'Import Riwayat'.
4. Backend menjalankan task `import\_historical\_trades` di background (Celery).
5. Task melakukan loop mundur per beberapa bulan memanggil `/fapi/v1/userTrades`.
6. Setiap fill yang ditemukan diproses: dimasukkan ke `exchange\_fills` (dengan deduplikasi), dipasangkan ke `trades` dengan `data\_source = 'historical\_import'`.
7. Kolom `psychology`, `trade\_setup\_tags`, dan `market\_context.bias\_arah\_manual` sengaja dibiarkan kosong.
8. Frontend menampilkan progress bar real-time (jumlah trade ditemukan, batch selesai).

5. Aktor

Trader

6. Hak Akses

Trader (Authenticated)

7. Input

- start_date (Date) - Validasi: Format YYYY-MM-DD, minimal setelah akun Binance dibuat (Wajib)
- end_date (Date) - Validasi: Format YYYY-MM-DD, maksimal hari ini (Wajib)

8. Output

Kompilasi ratusan trade baru di tabel trades, visualisasi progress bar di frontend.

9. Business Rules

- Data subjektif (setup, emosi) wajib kosong untuk mencegah bias hindsight.
- Trade hasil impor ditandai `data\_source = 'historical\_import'` dan otomatis dikecualikan dari kalkulasi statistik di Edge Discovery Engine (karena tidak memiliki tag setup).

10. Validasi

- Validasi rentang tanggal input tidak terbalik (start_date <= end_date).

- Deduplikasi otomatis fill dengan UNIQUE KEY `uq\_fill` di database.

11. Edge Cases

- Impor memakan waktu lama karena rate-limit API Binance. Solusi: jalankan secara bertahap (chunking) dan beri jeda (sleep) antar panggilan API.

12. Error Handling

- Jika token API expired/rate-limit terlampaui, jeda proses impor dan lanjutkan setelah rate-limit reset.
- Catat total trade sukses dan gagal di log akhir.

13. Database

- Mengisi tabel `trades` (`data\_source`='historical_import'), `exchange\_fills`, dan `trade\_fills`.

14. API

- GET /fapi/v1/userTrades
- GET /fapi/v1/income (untuk mencocokkan fee/funding)

15. UI/UX

- Frontend (src/pages/ImportWizard.jsx): Halaman Wizard khusus berisi rentang tanggal picker, tombol submit, progress bar meluncur (menggunakan websocket/SSE updates), dan panel hasil ringkasan (Mockup 13.9).

16. Security

Akses dibatasi hanya untuk Trader terautentikasi.

17. Dependencies

Fitur 1 (Autentikasi) dan Fitur 5 (Trade Collection).

18. Acceptance Criteria

- Seluruh trade lama dalam rentang tanggal berhasil diimpor tanpa duplikasi.
- Trade hasil impor muncul di Jurnal dengan label 'Import' (Mockup 13.2) dan tidak merusak angka expectancy di Edge Explorer.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur WIZARD IMPOR HISTORIS (Historical Import Wizard) sesuai mockup 13.9.

Tujuan fitur:

Menyediakan antarmuka wizard impor batch untuk menarik riwayat trade Binance masa lalu secara kronologis, memproses data objektif, serta memisahkannya dari penghitungan statistik edge agar bebas dari hindsight bias.

Fungsi utama:

- Menyediakan formulir tanggal picker di frontend.
- Menjalankan Celery background job untuk menarik data secara bertahap (pagination mundur).
- Melakukan pemrosesan dan deduplikasi fill transaksi Binance.
- Mengirimkan update persentase progress impor via WebSocket ke UI.

Alur bisnis:

1. Pengguna membuka `src/pages/ImportWizard.jsx` di frontend, memilih rentang tanggal, dan mengklik 'Import Riwayat'.
2. Frontend mengirimkan request ke POST `/api/v1/import/binance`.
3. Backend memverifikasi request dan memicu task Celery `import\_historical\_trades\_task(start\_ts, end\_ts)`.
4. Task membagi rentang tanggal menjadi potongan-potongan bulanan. Untuk setiap potongan, panggil GET `/fapi/v1/userTrades` dari Binance Futures.
5. Loop setiap fill transaksi:
 - a. Lakukan upsert ke tabel `exchange\_fills`. Jika fill sudah ada (dideteksi via `uq\_fill` constraint), lewati.
 - b. Kelompokkan fill berdasarkan order pembukaan dan penutupan yang berpasangan.
 - c. Masukkan data ke tabel `trades` dengan status lengkap, isi kolom `data\_source = 'historical\_import'`.
 - d. Biarkan data subjektif (tabel `psychology` dan `trade\_setup\_tags`) kosong.
 - e. Hubungkan fill ke `trade\_fills`.
6. Selama proses berjalan, task Celery mengirim pesan berkala via WebSocket berisi status progres saat itu.
7. Frontend memperbarui progress bar visual dan setelah selesai menampilkan log statistik impor (jumlah trade, fill, duplikat yang dilewati).

Validasi:

- Rentang tanggal tidak boleh kosong atau bernilai negatif.
- Kolom data subjektif wajib dikosongkan saat entri trade impor dibuat di DB.

Hak akses:

- Hanya Trader terautentikasi (JWT).

Kondisi gagal & Penanganan Error:

- Binance API Rate limit tercapai: Jeda task Celery selama 5 detik, lalu coba lagi.
- WebSocket terputus saat proses berjalan: Proses impor di backend tetap berjalan, frontend melakukan auto-reconnect untuk memuat status terbaru.

Integrasi:

- Input: JSON payload rentang tanggal.
- Output: WebSocket broadcast data, record baru di DB.
- Konfigurasi: Batch size, request rate-limit.
- Audit/Penyimpanan: Tabel `trades`, `trade\_fills`, `exchange\_fills`.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Risiko: Binance limitasi userTrades maksimal 7 hari per panggilan jika tidak memfilter simbol. Lakukan loop per simbol yang aktif atau gunakan rentang waktu pendek secara sekuensial.

FITUR 10 - LAYANAN SNAPSHOT EKUITAS (EQUITY SNAPSHOT SERVICE)

A. Ringkasan Fitur

Fitur ini melakukan pengambilan data saldo akun riil dan PnL belum terealisasi dari Binance Futures secara berkala (tiap jam) via Celery Beat, mendeteksi transfer (deposit/withdrawal) terpisah, dan menyajikan visualisasi kurva pertumbuhan akun riil.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Layanan Snapshot Ekuitas

2. Tujuan

Memantau pertumbuhan saldo akun riil (equity curve) dari waktu ke waktu secara akurat.

3. Deskripsi

Layanan background yang mencatat saldo akun berkala ke tabel equity_snapshots dan mendeteksi transaksi deposit/withdrawal agar tidak dianggap sebagai keuntungan/kerugian trading.

4. Business Flow

1. Celery Beat memicu task `capture\_equity\_snapshot` setiap jam.
2. Task memanggil GET /fapi/v2/balance dari Binance untuk mengambil saldo saat ini (balance) dan unrealized PnL.
3. Simpan data ke tabel `equity\_snapshots`.
4. Task kedua mendeteksi transfer dana masuk/keluar secara berkala melalui GET /fapi/v1/income dengan parameter `incomeType = TRANSFER`.
5. Jika ditemukan transfer baru, simpan ke tabel `account\_transfers`.
6. Dasbor membaca kedua tabel untuk memplot kurva ekuitas riil (Saldo Riil = Saldo Terakhir - Total Transfer).

5. Aktor

Equity Snapshot Service (Automated)

6. Hak Akses

Sistem (Background Service)

7. Input

8. Output

Record baru di tabel `equity\_snapshots` and `account\_transfers`.

9. Business Rules

- Snapshot saldo diambil secara berkala tanpa memandang apakah trader sedang aktif trading atau tidak.
- Transaksi transfer (deposit/withdrawal) dideteksi secara eksplisit untuk menjaga keakuratan grafik performa trading.

10. Validasi

Pastikan data saldo bernilai positif (balance >= 0).

11. Edge Cases

- Binance API tidak merespon saat jam sibuk. Solusinya: coba kembali (retry) setelah 5 menit.

12. Error Handling

- Abaikan snapshot jika terjadi kesalahan koneksi tunggal, namun catat di log audit.

13. Database

- Tabel `equity\_snapshots` (id, balance, unrealized_pnl, captured_at).
- Tabel `account\_transfers` (id, amount, asset, occurred_at, binance_transfer_ref).

14. API

- GET /fapi/v2/balance (Binance account balance)
- GET /fapi/v1/income (Binance income history)

15. UI/UX

- Frontend (src/pages/Dashboard.jsx): Grafik Kurva Ekuitas di Dasbor menampilkan dua garis berdampingan: R-kumulatif (murni kualitas keputusan trading) dan Saldo Real (pertumbuhan saldo riil setelah memperhitungkan transfer) (Mockup 13.4).

16. Security

Memerlukan API Key Binance dengan izin pembacaan akun (read-only).

17. Dependencies

Fitur 2 (Binance Sync) untuk infrastruktur koneksi API.

18. Acceptance Criteria

- Data saldo akun tersimpan otomatis setiap jam.
- Deposit dan withdrawal terdeteksi dan tercatat terpisah dengan benar.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur LAYANAN SNAPSHOT EKUITAS (Equity Snapshot Service).
Tujuan fitur:
Mencatat data pertumbuhan saldo akun Binance secara real-time dan melacak transaksi transfer eksternal secara terpisah untuk menyajikan grafik kurva pertumbuhan ekuitas yang murni.
Fungsi utama:
- Mengambil saldo akun dan PnL belum terealisasi dari Binance API `/fapi/v2/balance` setiap jam.
- Mendeteksi transaksi deposit dan penarikan (transfer) dari Binance `/fapi/v1/income`.
- Menyimpan snapshot saldo ke database MySQL.
- Menyediakan dataset grafik garis ganda di API endpoint untuk frontend.
Alur bisnis:
1. Task Celery Beat `capture\_equity\_snapshot` berjalan setiap jam.
2. Panggil GET `/fapi/v2/balance` dari Binance, ambil nilai saldo saat ini dan unrealized PnL, lalu simpan ke tabel `equity\_snapshots`.
3. Task periodik kedua `detect\_account\_transfers` berjalan harian:
 a. Panggil GET `/fapi/v1/income` dengan filter `incomeType=TRANSFER` untuk simbol koin terdaftar.
 b. Untuk setiap data transfer, lakukan pengecekan duplikasi berdasarkan reference transaction ID Binance.
 c. Simpan transfer dana (nilai positif untuk deposit, negatif untuk withdrawal) ke tabel `account\_transfers`.
4. Endpoint GET `/api/v1/analytics/equity-curve` dipanggil oleh frontend untuk menyusun grafik ekuitas riil.
Validasi:
- Saldo akun tidak boleh bernilai negatif.
- Data transfer harus disaring dengan benar agar transfer internal antar wallet Binance tidak tercatat sebagai deposit.
Hak akses:
- Komponen backend internal (scheduler).
Kondisi gagal & Penanganan Error:
- Jika server API Binance tidak merespon saat pengambilan data per jam, lewati snapshot tersebut (jangan isi data kosong atau nol) dan catat warning di log.
Integrasi:
- Input: Binance API endpoints.
- Output: Update tabel `equity\_snapshots` dan `account\_transfers`.
- Konfigurasi: Waktu pemicuan scheduler di Celery.
- Audit/Penyimpanan: Tabel snapshot ekuitas.
Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.
Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Optimasi: Gunakan kalkulasi di level database atau pandas pada API response untuk mempercepat rendering kurva ekuitas di frontend.

FITUR 11 - MESIN ANALITIS (ANALYTICS ENGINE)

A. Ringkasan Fitur

Mesin komputasi berbasis pandas/NumPy yang menghitung metrik performa trading utama (Win Rate, Average RR, Expectancy, Profit Factor, Max Drawdown, MFE/MAE, Recovery Factor, Average Holding Time) serta metrik deskriptif margin/equity.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Mesin Analitis

2. Tujuan

Menyediakan metrik statistik performa trading yang komprehensif bagi trader.

3. Deskripsi

Komponen pemrosesan data analitik menggunakan Python pandas untuk menghasilkan data agregat performa trade.

4. Business Flow

1. Trader membuka Dasbor atau halaman Jurnal.
2. Frontend memanggil API analitik dengan parameter filter (rentang tanggal, simbol, sesi, kondisi market, dll.).
3. Backend mengambil data trade terkait dari database MySQL.
4. Backend mengonversi data ke pandas DataFrame.
5. Backend menghitung seluruh metrik performa.
6. Hasil kalkulasi dikembalikan sebagai JSON response ke frontend.

5. Aktor

Trader

6. Hak Akses

Trader (Authenticated)

7. Input

- filter_pair (String) - Validasi: Opsional, filter per pair (Opsional)
- filter_session (String) - Validasi: Opsional, filter per sesi (Opsional)
- filter_source (String) - Validasi: 'live' (hanya trade bertag) atau 'all' (termasuk import) (Opsional)

8. Output

Kumpulan angka metrik performa (Win Rate, Expectancy, Max Drawdown, dsb) dalam tipe data Decimal.

9. Business Rules

- Semua kalkulasi finansial harus menggunakan kelas `Decimal` Python untuk menghindari precision error.
- Metrik deskriptif baru (v1.3): Return on Margin (PnL / margin) dan Dampak terhadap Equity (PnL / saldo entry) dihitung secara on-the-fly.
- Mesin Analitis menyediakan filter khusus untuk memisahkan data impor historis agar tidak mengaburkan metrik headline.

10. Validasi

Validasi input filter tanggal tidak melebihi hari ini.

11. Edge Cases

- Jumlah trade masih nol. Solusinya: kembalikan metrik dengan nilai default 0 tanpa melempar error pembagian dengan nol (division by zero).

12. Error Handling

- Tangkap error konversi tipe data.
- Kembalikan HTTP 400 jika format filter salah.

13. Database

- Membaca tabel `trades` dan `equity\_snapshots`.

14. API

- GET /api/v1/analytics/summary (Menghasilkan ringkasan metrik)
- GET /api/v1/analytics/distribution (Menghasilkan data distribusi R-multiple)

15. UI/UX

- Frontend (src/pages/Dashboard.jsx & src/pages/Journal.jsx): Menampilkan metrik headline di kartu khusus dasbor, serta menyediakan switch/toggle 'Trade Bertag Saja' vs 'Semua Trade' (Mockup 13.2 & 13.4).
- Grafik histogram distribusi performa trade.

16. Security

Otentikasi JWT.

17. Dependencies

Fitur 5 (Trade Collection) untuk data masukan trade.

18. Acceptance Criteria

- Perhitungan expectancy sesuai dengan rumus: $(\text{Win Rate} * \text{Average Win}) - (\text{Loss Rate} * \text{Average Loss})$.
- Filter data berfungsi dengan benar dan responsif (< 1 detik).

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur MESIN ANALITIS (Analytics Engine).

Tujuan fitur:

Memproses data transaksi mentah menggunakan pustaka Pandas/NumPy secara efisien untuk menyajikan analisis performa statistik trading (seperti Win Rate, Expectancy, Profit Factor) kepada pengguna.

Fungsi utama:

- Memuat data transaksi `trades` dan data saldo `equity\_snapshots` ke dalam struktur Pandas DataFrame.
- Menghitung metrik performa dasar dan tingkat lanjut (Win Rate, Average RR, Expectancy, Profit Factor, Max Drawdown, MFE/MAE, Recovery Factor, Average Holding Time).
- Menghitung parameter Return on Margin dan Dampak terhadap Equity secara dinamis.
- Mendukung filter data per dimensi (symbol, sesi, volatilitas, rentang waktu).

Alur bisnis:

1. Frontend memicu pemanggilan REST API GET `/api/v1/analytics/summary` dengan filter parameter.
2. Service backend memuat data trade relevan dari DB dan mengubahnya menjadi DataFrame Pandas.
3. Jalankan kalkulasi:
 - a. Win Rate = jumlah trade profit / total trade.
 - b. Expectancy = (Win Rate * Average Win R) - (Loss Rate * Average Loss R).
 - c. Profit Factor = total profit R / total loss R.
 - d. Max Drawdown = penurunan puncak ekuitas tertinggi ke lembah terendah secara persentase.
 - e. Return on Margin = realized PnL / margin trade.
 - f. Dampak Equity = realized PnL / saldo snapshot saat entry.
4. Format seluruh hasil metrik ke dalam tipe data Decimal dengan pembulatan 4 angka desimal.
5. Kembalikan data JSON ke frontend React `src/pages/Dashboard.jsx`.

Validasi:

- Data input filter tanggal harus berada dalam rentang logis dan aman.
- Hindari division by zero error dengan memasang pengaman jika total trade atau total loss bernilai nol.

Hak akses:

- Trader terautentikasi (JWT).

Kondisi gagal & Penanganan Error:

- Jika query database lambat, terapkan pagination di tingkat database atau buat agregasi cache redis berkala.

Integrasi:

- Input: Parameter filter dari frontend React.
- Output: JSON berisi objek metrik analitis lengkap.
- Konfigurasi: Redis caching timeout.
- Audit/Penyimpanan: Database MySQL.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Optimasi: Gunakan query terindeks untuk mengambil kolom minimal (id, pair, entry_time, exit_time, pnl, fee, margin, risk_amount, rr_realized) untuk mempercepat load data.

FITUR 12 - MESIN PENEMU EDGE (EDGE DISCOVERY ENGINE)

A. Ringkasan Fitur

Fitur analitik canggih yang mencari kombinasi setup dan kondisi pasar terbaik dengan expectancy positif tertinggi menggunakan Bootstrap resampling (10.000 iterasi) untuk interval kepercayaan, Wilson Score untuk win rate, dan koreksi Benjamini-Hochberg FDR.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Mesin Penemu Edge

2. Tujuan

Menemukan kombinasi setup dan kondisi pasar secara objektif yang memiliki keunggulan statistik nyata (edge).

3. Deskripsi

Job batch harian (Celery Beat) yang menganalisis seluruh kombinasi tag setup yang pernah muncul, menghitung expectancy, confidence interval, dan p-value untuk validasi.

4. Business Flow

1. Celery Beat memicu task `run\_edge\_discovery` setiap hari pada jam sepi.
2. Task mengambil seluruh data trade dengan status lengkap (locked_at IS NOT NULL dan data_source != 'historical_import').
3. Kelompokkan trade berdasarkan kombinasi tag setup yang muncul bersamaan (dibatasi maksimal 4 tag per kombinasi).
4. Saring kombinasi yang memiliki jumlah sampel $n \geq 20$.
5. Untuk setiap kombinasi:
 - a. Hitung rata-rata expectancy (R).
 - b. Lakukan Bootstrap resampling sebanyak 10.000 kali untuk menentukan 95% Confidence Interval (CI) expectancy (ambil persentil 2.5% dan 97.5%).
 - c. Hitung Wilson Score Interval untuk win rate.
 - d. Hitung p-value (probabilitas bahwa expectancy positif terjadi karena faktor kebetulan).
6. Terapkan FDR Correction (Benjamini-Hochberg) pada seluruh p-value kombinasi untuk menyaring penemuan palsu.
7. Bagi data secara kronologis (70% pertama untuk discovery/training, 30% sisa untuk validasi out-of-sample).
8. Simpan kombinasi yang lolos ke tabel `edge\_blueprints`.

5. Aktor

Edge Discovery Engine (Automated)

6. Hak Akses

Sistem (Background Service)

7. Input

8. Output

Kumpulan baris baru di tabel `edge\_blueprints` dengan data expectancy, batas bawah/atas CI, dan p-value.

9. Business Rules

- Hanya mengevaluasi trade dengan data_source = 'manual' atau 'binance_sync'. Trade impor historis dikecualikan.
- Sizing/leverage tidak melibatkan dalam perhitungan (murni menggunakan satuan R) untuk menjaga kemurnian evaluasi kualitas setup.

10. Validasi

- Validasi jumlah sampel $n \geq 20$ untuk kelayakan statistik awal.
- Validasi out-of-sample 70/30 untuk menghindari overfitting.

11. Edge Cases

- Variansi data sangat tinggi sehingga Bootstrap CI menghasilkan rentang yang sangat lebar. Solusinya: naikan status ke Validation hanya jika $n \geq 30$.

12. Error Handling

- Catat log kegagalan komputasi.
- Lewati kombinasi yang memiliki data tidak lengkap.

13. Database

- Membaca tabel `trades`, `trade\_setup\_tags`.
- Menulis ke tabel `edge\_blueprints`.

14. API

- GET /api/v1/edges/blueprints (Membaca blueprint edge hasil penemuan)

15. UI/UX

- Menampilkan daftar blueprint edge di halaman Edge Blueprint Explorer (Mockup 13.5).

16. Security

Akses baca dibatasi hanya untuk Trader terautentikasi.

17. Dependencies

Fitur 5 (Trade Collection) dan Fitur 6 (Trade Execution & Immutability).

18. Acceptance Criteria

- Proses Bootstrap resampling 10.000 kali selesai tanpa timeout di server.
- P-value dihitung secara benar dan koreksi Benjamini-Hochberg diterapkan dengan target FDR 5%.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur MESIN PENEMU EDGE (Edge Discovery Engine).

Tujuan fitur:
Mencari kombinasi setup dan kondisi pasar terbaik dengan keunggulan statistik (expectancy R positif) yang valid secara ilmiah menggunakan Bootstrap resampling 10.000 iterasi dan koreksi FDR Benjamini-Hochberg.

Fungsi utama:

- Mengelompokkan riwayat trade berdasarkan irisan kombinasi tag setup (maksimal 4 tag).
- Menjalankan simulasi Bootstrap resampling sebanyak 10.000 kali untuk menghasilkan batas Confidence Interval (CI) 95%.
- Mengkalkulasikan Wilson Score Interval untuk batas win rate.
- Melakukan koreksi multiple comparisons (FDR) menggunakan metode Benjamini-Hochberg.
- Memisahkan dataset 70/30 secara kronologis untuk validasi out-of-sample.

Alur bisnis:

1. Task Celery Beat ``run_edge_discovery_engine`` dijalankan secara berkala (harian).
2. Backend menarik data trade lengkap (``locked_at IS NOT NULL`` dan ``data_source != 'historical_import'``).
3. Cari seluruh kombinasi tag setup yang unik yang benar-benar pernah muncul bersamaan di trade trader.
4. Saring kombinasi yang memiliki jumlah sampel trade $n \geq 20$.
5. Untuk setiap kombinasi yang lolos seleksi:
 - a. Urutkan trade secara kronologis. Bagi data menjadi 70% awal (Discovery) dan 30% sisa (Validation).
 - b. Di data Discovery, lakukan resampling acak dengan pengembalian sebanyak 10.000 kali. Pada setiap resample, hitung rata-rata R-multiple.
 - c. Tentukan 95% CI: ambil persentil ke-2.5 sebagai ``ci_lower`` dan persentil ke-97.5 sebagai ``ci_upper``.
 - d. Hitung p-value menggunakan one-sample t-test atau non-parametric bootstrap test (menguji apakah rata-rata $R > 0$).
 - e. Hitung batas bawah win rate menggunakan Wilson Score Interval.
6. Kumpulkan p-value dari semua kombinasi, lalu jalankan metode Benjamini-Hochberg FDR correction dengan target FDR 5% untuk menandai kombinasi mana yang memiliki signifikansi statistik nyata.
7. Simpan atau perbarui data ke tabel ``edge_blueprints`` di database MySQL.

Validasi:

- Hanya trade bertag lengkap yang digunakan (eksklusi data impor historis).
- Analisis statistik wajib dievaluasi murni dalam satuan R (expectancy R-multiple).

Hak akses:

- Komponen backend internal (background service).

Kondisi gagal & Penanganan Error:

- Jumlah total trade aktif kurang dari 20: Jeda operasi discovery, log info 'Data sampel tidak mencukupi'.

Integrasi:

- Input: Riwayat transaksi dari database.
- Output: Record baru atau ter-update di tabel ``edge_blueprints``.
- Konfigurasi: Jumlah iterasi bootstrap (10.000), target rate FDR (5%).
- Audit/Penyimpanan: Tabel ``edge_blueprints``.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Risiko komputasi: Bootstrap 10.000 iterasi untuk puluhan kombinasi tag dapat memakan beban CPU tinggi. Pastikan kode dioptimalkan menggunakan NumPy vectorized operations.

FITUR 13 - MESIN VALIDASI & PEMANTAU STATUS EDGE (EDGE VALIDATION & STATUS MONITOR)

A. Ringkasan Fitur

Fitur ini memantau status kematangan edge secara otomatis (Learning -> Research -> Validation -> Production -> Monitoring) berdasarkan volume trade dan kestabilan performa, serta memberikan peringatan dini jika performa edge Production memburuk.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Mesin Validasi & Pemantau Status Edge

2. Tujuan

Mengawasi siklus hidup (lifecycle) edge dan mendeteksi secara dini jika suatu strategi trading mulai kehilangan keunggulannya di pasar.

3. Deskripsi

Modul transisi status otomatis untuk edge_blueprints dan sistem deteksi penurunan performa berbasis pemantauan run-rate.

4. Business Flow

- Setiap kali job discovery harian berjalan, sistem mengevaluasi kriteria transisi status tiap blueprint:
 - $n < 20$ -> Status: Learning.
 - $20 \leq n < 30$ -> Status: Research.
 - $n \geq 30$, tiga kriteria (stabil, berulang, robust) belum terpenuhi semua -> Status: Validation.
 - $n \geq 50-100$, lolos tiga kriteria + signifikan setelah FDR correction -> Status: Production.
- Untuk edge berstatus Production, sistem melakukan perbandingan performa terbaru:
 - Ambil rata-rata R-multiple dari 30 trade terakhir untuk kombinasi tag tersebut.
 - Bandingkan dengan batas bawah CI (`ci_lower`) historisnya yang tersimpan di ``edge_blueprints``.
 - Jika rata-rata 30 trade terakhir berada di bawah ``ci_lower``:
 - Turunkan status edge menjadi 'Monitoring'.
 - Pemicuan Notification Service untuk mengirimkan alert 'status edge turun' ke trader.

5. Aktor

Sistem (Automated)

6. Hak Akses

Sistem (Background Service)

7. Input

8. Output

Pembaruan status di tabel ``edge_blueprints``, pemicuan alert notifikasi.

9. Business Rules

- Kriteria transisi status didasarkan pada data statistik objektif, bukan pilihan manual trader.
- Edge yang berada di status Monitoring tidak boleh dijadikan acuan keputusan entry utama di dasbor.
- Jika performa membaik kembali ke dalam batas CI historis, status dapat naik kembali ke Production secara otomatis.

10. Validasi

Validasi kelengkapan data trade pendukung sebelum mengubah status.

11. Edge Cases

- Edge di status Monitoring terus memburuk. Sistem akan mempertahankan status di Monitoring (tidak menghapus data secara otomatis agar histori tetap tersimpan untuk review).

12. Error Handling

- Catat setiap perubahan status di audit log.
- Jika gagal mengirim notifikasi, tetap lakukan perubahan status di database.

13. Database

- Update tabel `edge\_blueprints` (status, updated_at).

14. API

- GET /api/v1/edges/status (Membaca status kematangan edge)

15. UI/UX

- Frontend (src/pages/EdgeDetail.jsx): Indikator status (badge berwarna) di samping nama edge di dasbor (Mockup 13.5 & 13.6).
- Detail grafik run-rate performa terakhir dibandingkan dengan batas CI historis.

16. Security

Otentikasi JWT.

17. Dependencies

Fitur 8 (Notification Service) dan Fitur 12 (Edge Discovery Engine).

18. Acceptance Criteria

- Status otomatis turun ke Monitoring jika performa 30 trade terakhir jatuh di bawah ci_lower.
- Notifikasi berhasil terkirim ke email dan web push ketika transisi penurunan status terjadi.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur MESIN VALIDASI & PEMANTAU STATUS EDGE (Edge Validation & Status Monitor).

Tujuan fitur:

Mengelola transisi status kematangan edge secara otomatis (Learning -> Research -> Validation -> Production -> Monitoring) dan mendeteksi degradasi performa edge secara dini menggunakan pemantauan berbasis batas bawah historis (Confidence Interval).

Fungsi utama:

- Menerapkan aturan transisi status berdasarkan ukuran sampel (n) dan kriteria statistik.
- Melacak performa run-rate terbaru (30 trade terakhir) untuk edge berstatus Production.
- Menurunkan status edge ke 'Monitoring' jika performa terdegradasi.
- Memanggil Notification Service untuk mengirim peringatan multi-saluran ke trader.

Alur bisnis:

1. Task periodik harian memicu evaluasi status untuk setiap entri di `edge\_blueprints`.
2. Tentukan status baru:
 - a. Jika sampel `n < 20`: status = `learning`.
 - b. Jika `20 <= n < 30`: status = `research`.
 - c. Jika `n >= 30` tapi belum lolos pengujian signifikansi: status = `validation`.
 - d. Jika `n >= 50` dan lolos 3 kriteria validasi (stabilitas periode, konsistensi lintas pair, robustness parameter) + signifikan setelah FDR: status = `production`.
3. Untuk blueprint yang berstatus `production`:
 - Ambil 30 trade terbaru yang menggunakan kombinasi tag tersebut.
 - Hitung rata-rata R-multiple dari 30 trade tersebut.
 - Bandingkan dengan nilai `ci\_lower` historis yang tersimpan di baris blueprint.
 - Jika rata-rata R-multiple < `ci\_lower`:
 - a. Update status blueprint di DB menjadi `monitoring`.
 - b. Kirim notifikasi 'edge_status_change' via email, push notification, dan in-app banner.

Validasi:

- Transaksi penurunan status harus idempotent dan dicatat di log audit.
- Penghitungan rata-rata 30 trade terakhir hanya boleh menyertakan trade bertag lengkap.

Hak akses:

- Internal backend (Celery worker).

Kondisi gagal & Penanganan Error:

- Data trade pendukung dihapus: Terapkan constraint integrity di DB, log warning jika jumlah sampel di bawah threshold.

Integrasi:

- Input: Data dari database MySQL.
- Output: Update data tabel `edge\_blueprints`, event trigger ke Notification Service (Fitur 8).
- Konfigurasi: Parameter batas sampel (20, 30, 50), run-rate sample size (30).
- Audit/Penyimpanan: Tabel `edge\_blueprints`.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Risiko: Perubahan regime market secara mendadak dapat menurunkan performa banyak edge sekaligus. Pastikan sistem pemantau status berjalan harian agar trader mendapatkan info pelemahan secepatnya.

FITUR 14 - ASISTEN AI (AI COACH SERVICE)

A. Ringkasan Fitur

Fitur asisten cerdas yang memberikan evaluasi kontekstual pasca-trade dengan membandingkan trade yang baru selesai dengan histori setup serupa di masa lalu menggunakan data teragregasi yang dianonimkan demi privasi.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Asisten AI

2. Tujuan

Membantu trader mengidentifikasi kesalahan emosional atau deviasi rencana melalui feedback kontekstual dari kecerdasan buatan.

3. Deskripsi

Integrasi LLM API yang membaca data trade dan memberikan ulasan kualitatif personal tanpa mengirimkan data identitas akun yang sensitif.

4. Business Flow

1. Trade selesai dan berstatus lengkap.
2. Trader menekan tombol 'Minta Evaluasi AI' di halaman Trade Detail (Mockup 13.3).
3. Backend mengumpulkan data trade terkait (setup, psikologi, market context, realized RR).
4. Backend mengambil ringkasan historis (win rate, average RR) dari trade lain dengan setup serupa.
5. Backend menganonimkan data (menghapus saldo akun mentah, ID API, dll).
6. Mengirimkan prompt kontekstual ke LLM (seperti OpenAI GPT atau self-hosted LLM lokal).
7. LLM mengembalikan ulasan analitis.
8. Ulasan disimpan di database dan ditampilkan pada kartu AI Coach di halaman detail.

5. Aktor

Trader

6. Hak Akses

Trader (Authenticated)

7. Input

- trade_id (UUID) - Validasi: Valid UUID di tabel trades (Wajib)

8. Output

Teks evaluasi kualitatif dari AI Coach.

9. Business Rules

- Data saldo akun dan detail kunci API dilarang keras dikirim ke LLM eksternal.
- Sistem harus menyediakan opsi untuk menggunakan LLM lokal (seperti Llama via Ollama) bagi pengguna yang menginginkan privasi data 100%.

10. Validasi

Validasi bahwa trade terkait telah berstatus lengkap (exit_time tidak NULL).

11. Edge Cases

- API LLM eksternal limit / timeout. Solusinya: tampilkan pesan error ramah dan sediakan tombol retry.

12. Error Handling

- Tangkap kegagalan koneksi API LLM dan kembalikan HTTP 503 Service Unavailable dengan deskripsi yang jelas.

13. Database

- Menyimpan ulasan ke tabel `psychology` (kolom baru / update free_notes) atau tabel asisten khusus (opsional). Rencana saat ini: disimpan di tabel `psychology` pada kolom terpisah (atau disajikan dinamis).

14. API

- POST /api/v1/ai-coach/review (Request: trade_id; Response: review_text)

15. UI/UX

- Frontend (src/pages/TradeDetail.jsx): Kartu khusus 'AI COACH' di halaman Trade Detail dengan teks feedback terformat indah (Mockup 13.3). Terdapat tombol 'Minta Evaluasi AI' yang menampilkan spinner loading saat proses analisis berlangsung.

16. Security

Anonimisasi data sebelum dikirim keluar dari server TEIS.

17. Dependencies

Fitur 11 (Analytics Engine) untuk mendapatkan metrik setup serupa.

18. Acceptance Criteria

- Feedback AI berhasil dibuat dan menampilkan perbandingan relevan dengan trade serupa di masa lalu.
- Tidak ada data sensitif akun yang bocor ke log API eksternal.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun fitur ASISTEN AI (AI Coach Service).

Tujuan fitur:

Menyediakan modul asisten kecerdasan buatan (LLM) yang menghasilkan analisis kualitatif dan feedback pasca-transaksi untuk membantu mengoreksi bias emosional trader tanpa membocorkan data saldo akun sensitif.

Fungsi utama:

- Mengumpulkan parameter kualitatif dan kuantitatif dari trade terkait.
- Menganonimkan data (membersihkan saldo akun, API credentials, dll.).
- Mengambil data ringkasan historis untuk trade dengan setup serupa.
- Membangun prompt terstruktur dan memanggil API LLM (OpenAI GPT atau Ollama lokal).
- Menyimpan teks tanggapan ke kolom database dan merendernya di UI.

Alur bisnis:

1. Trader membuka halaman detail trade `src/pages/TradeDetail.jsx` dan mengklik tombol 'Minta Evaluasi AI' pada kartu AI Coach.
2. Frontend mengirimkan request ke POST `/api/v1/ai-coach/review` dengan payload `trade\_id`.
3. Backend memverifikasi otentikasi, mengambil data dari tabel `trades`, `psychology`, `market\_context`, dan `trade\_execution` untuk trade_id tersebut.
4. Panggil modul analitik untuk mengambil performa historis dari trade lain yang menggunakan setup yang sama (ambil win rate, expectancy, total trade).
5. Bentuk prompt teks terstruktur. ****Perhatian****: Jangan pernah menyertakan saldo akun riil, leverage, margin USD, atau detail kunci API.
6. Kirim prompt ke LLM API terkonfigurasi.
7. Simpan teks ulasan hasil respon LLM ke database (tabel `psychology` atau tabel asisten khusus).
8. Kembalikan ulasan teks tersebut ke UI frontend untuk dirender secara dinamis.

Validasi:

- Verifikasi bahwa trade sudah ditutup (exit_time tidak NULL).
- Validasi parameter LLM (temperature, token limit) agar respon terfokus pada analisis trading.

Hak akses:

- Hanya Trader terautentikasi (JWT).

Kondisi gagal & Penanganan Error:

- API LLM timeout / rate limit: Kirim HTTP 503 Service Unavailable, tampilkan pesan warning ramah di UI, jangan buat crash backend.

Integrasi:

- Input: JSON payload via REST API.
- Output: Teks tanggapan AI Coach.
- Konfigurasi: API key OpenAI, host URL Ollama, LLM Model (.env).
- Audit/Penyimpanan: Simpan tanggapan ke database MySQL.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Pengembangan masa depan: Layanan ini dapat ditingkatkan dengan RAG (Retrieval-Augmented Generation) menggunakan buku jurnal mingguan trader sebagai referensi tambahan bagi AI.

FITUR 15 - DASBOR & EXPLORER CETAK BIRU EDGE (DASHBOARD & EDGE BLUEPRINT EXPLORER)

A. Ringkasan Fitur

Fitur visualisasi utama berupa dasbor ringkasan performa trading (equity curve dua garis, win rate, profit factor), tabel blueprint edge dengan status kematayannya, halaman detail edge, serta galeri review mingguan.

B. Dokumentasi Lengkap Fitur

1. Nama Fitur

Dasbor & Explorer Cetak Biru Edge

2. Tujuan

Menjadi antarmuka utama bagi trader untuk mengevaluasi hasil trading dan mengawasi edge mereka.

3. Deskripsi

Halaman-halaman web frontend (React) yang menyajikan metrik analitik dasar, kurva ekuitas, dan detail blueprint edge secara visual.

4. Business Flow

1. Trader membuka web app.
2. Dasbor Utama (Mockup 13.4) menampilkan metrik expectancy, win rate, profit factor, max drawdown, grafik coverage market, dan equity curve dua garis (R-kumulatif vs Saldo Real).
3. Trader berpindah ke menu Edge Blueprint (Mockup 13.5) untuk melihat tabel seluruh kombinasi edge hasil penemuan sistem beserta status kematangannya (Learning/Research/Validation/Production/Monitoring).
4. Trader mengklik salah satu edge untuk membuka Edge Detail (Mockup 13.6) guna melihat checklist validasi (stabilitas, repeatabilitas, robustness) dan daftar trade pendukung.
5. Trader membuka menu Review (Mockup 13.7) untuk review mingguan berupa galeri screenshot chart dan catatan bebas.

5. Aktor

Trader

6. Hak Akses

Trader (Authenticated)

7. Input

- filter_source (String) - Validasi: 'live' atau 'all' (Opsional)
- filter_timeframe (String) - Validasi: 30_days, 90_days, all (Opsional)

8. Output

Tampilan UI interaktif di web browser.

9. Business Rules

- Dasbor harus menyediakan toggle filter 'Semua Trade' vs 'Trade Bertag Saja' agar data impor tidak merusak statistik keputusan trading.
- Kurva ekuitas riil harus memperhitungkan data deposit/withdrawal dari account_transfers.

10. Validasi

Validasi token JWT sebelum memuat data halaman.

11. Edge Cases

- Resolusi layar sangat kecil (mobile). Solusinya: grafik dan tabel dikonfigurasi responsif (scrollable horizontal atau berubah menjadi list card).

12. Error Handling

- Tampilkan state loading yang indah dan penanganan fallback jika API gagal mengembalikan data.

13. Database

- Membaca seluruh tabel utama di database.

14. API

- Mengintegrasikan seluruh GET API dari modul analitik, notifikasi, dan edge blueprints.

15. UI/UX

- Tema gelap premium (dark mode) dengan warna latar violet gelap dan aksen teal.

- Visualisasi grafik menggunakan Chart.js, Recharts, atau ApexCharts.

- Navigasi sidebar yang rapi (Mockup 13.1 - 13.9).

- Berkas frontend utama: `src/pages/Dashboard.jsx`, `src/pages/EdgeExplorer.jsx`, `src/pages/EdgeDetail.jsx`, `src/pages/WeeklyReview.jsx`, `src/pages/Journal.jsx`, `src/pages/TradeDetail.jsx`.

16. Security

Seluruh komunikasi data wajib menggunakan HTTPS.

17. Dependencies

Seluruh fitur backend (Fitur 1 s.d. 14) harus sudah mengimplementasikan API-nya masing-masing.

18. Acceptance Criteria

- Halaman dasbor dimuat dalam waktu < 1.5 detik.

- Grafik R-kumulatif dan Saldo Real terplot berdampingan dengan benar.

- Navigasi antar halaman berjalan lancar tanpa kehilangan state otentikasi.

C. Prompt Agentic AI Untuk Implementasi Fitur

[INSTRUKSI AI] Salin prompt di bawah ini untuk digunakan oleh AI Coding Agent Anda.

Anda adalah Senior Software Engineer. Bangun antarmuka DASBOR UTAMA & EXPLORER CETAK BIRU EDGE (Dashboard & Edge Explorer UI) sesuai mockup 13.4, 13.5, 13.6, dan 13.7.

Tujuan fitur:

Menyediakan antarmuka visual (React web app) bertema gelap premium untuk dasbor analitik utama, tabel blueprint edge interaktif, detail edge lengkap, dan galeri review mingguan.

Fungsi utama:

- Merender navigasi sidebar responsif.
- Menampilkan grafik garis ganda ekuitas (R-kumulatif vs Saldo Real).
- Menampilkan tabel blueprint edge yang dapat diurutkan berdasarkan parameter.
- Merender panel validasi kelayakan kriteria edge.
- Menyediakan galeri review mingguan berisi screenshot chart dan catatan.

Alur bisnis:

1. Rancang navigasi sidebar di `src/components/Sidebar.jsx` (Journal, Dashboard, Edge Blueprint, Review) yang terintegrasi dengan `react-router-dom`.
2. Halaman Dasbor Utama (`src/pages/Dashboard.jsx`):
 - a. Tampilkan kartu metrik headline (Expectancy, Win Rate, Profit Factor, Max Drawdown, Avg Return on Margin) berdasarkan respons API analitik (Fitur 11).
 - b. Rendeng grafik garis ganda Recharts: R-kumulatif (warna teal) berdampingan dengan Saldo Real (warna hijau) (Fitur 10).
 - c. Tampilkan diagram lingkaran 'Coverage' kondisi pasar (Mockup 13.4).
 - d. Sediakan switch filter 'Trade Bertag Saja' vs 'Semua Trade' di navbar atas.
3. Halaman Edge Explorer (`src/pages/EdgeExplorer.jsx`):
 - Render tabel blueprint edge dengan kolom: Edge, Status (badge berwarna), n, Expectancy (CI 95%), dan Trend (Mockup 13.5).
4. Halaman Edge Detail (`src/pages/EdgeDetail.jsx`):
 - Tampilkan nama edge, pill tags setup, grafik batang range Confidence Interval (CI), kartu checklist validasi (Stabilitas, Repeatabilitas, Robustness), dan daftar trade pendukung (Mockup 13.6).
5. Halaman Weekly Review (`src/pages/WeeklyReview.jsx`):
 - Kelompokkan trade per minggu, tampilkan galeri tangkapan layar chart yang diunggah dan catatan kualitatif (Mockup 13.7).

Validasi:

- Izin akses halaman harus dilindungi oleh AuthContext JWT di frontend.
- Format representasi angka di UI harus konsisten dibatasi hingga 2 angka desimal (kecuali harga kripto).

Hak akses:

- Trader terotentikasi.

Kondisi gagal & Penanganan Error:

- Panggilan API gagal: Tampilkan state loading skeleton dan pesan error 'Gagal memuat data, coba lagi'.

Integrasi:

- Input: API responses dari backend.
- Output: Antarmuka UI interaktif di browser.
- Konfigurasi: Vite config, env var backend API URL.
- Audit/Penyimpanan: Client-side storage (localStorage) untuk JWT token.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready.

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

D. Catatan Implementasi

Aksen Desain: Sesuai dengan panduan Web Application Development, pastikan antarmuka bersih dari placeholder, menggunakan transisi hover mikro-animasi pada tombol/baris tabel, dan tipografi modern (seperti font Inter).